

第三章 需求描述与规约

需求描述及其严格定义是软件系统构建过程的首要环节。在这一阶段将得到需求分析的关键制品(artefact),包括领域模型与用例模型。其中,领域模型即概念(类)模型,本质上是仅包含类属性及其职责描述、不包含类方法的类图。用例模型则包含以下制品。

(1) 用例图:用于从整体上展现系统应提供的功能及性能^①,以及系统运行所需的外部环境。其中,外部环境主要指系统的外部参与者。

(2) 用例文本:用于描述特定参与者使用系统功能执行其业务并获得其所期望结果的完整过程,应包含对失败和成功情形的刻画。

(3) 系统序列图:用于刻画在某个用例的语境(context)中,参与者和系统之间的交互序列;即描述参与者向系统提供数据并发出请求,而由系统处理请求后返回响应,以实现参与者期望的业务目标。

(4) 系统操作及操作契约:系统操作主要刻画系统在某个用例的语境中所提供的服务,它来自系统序列图中系统所接收并处理的来自参与者的请求。操作契约通过前置条件和后置条件的方式,描述某个用例的语境中发生某个系统操作所引发的系统状态变化,而系统状态变化则通过对象、对象属性及对象间关系的变化来反映。

本章将主要以一个比较典型的 Web 应用开源项目 JPetStore 作为案例进行介绍,其中穿插使用了人们日常生活和学习中常常接触到的银行 ATM 系统、教室设备管理系统等进行相关技术的辅助阐释,以方便读者理解和掌握。本章会细致地介绍软件系统需求分析阶段各个关键制品的详细开发过程,并描述各制品之间的前驱后继关系以及相互约束,以便读者理解和把握需求分析的本质。

【学习目标】

(1) 理解用例、类、系统操作、系统操作契约等概念。

(2) 具备需求建模的能力,系统掌握用例模型构建方法(即①~②所描述的能力)、概念模型构建方法(即③所描述的能力)、行为模型构建方法(即④~⑤所描述的能力):① 能够从获取的需求描述中发现和识别用例,并使用 UML 构造用例图;② 能够对用例进行较为系统和结构化的文字描述,并开发高层文本用例

^① 根据 ISO/IEC 25010 的质量属性系统,需求可分为功能性、性能效率、兼容性、易用性等类型。普遍的做法是简单划分为功能性和非功能性两类。

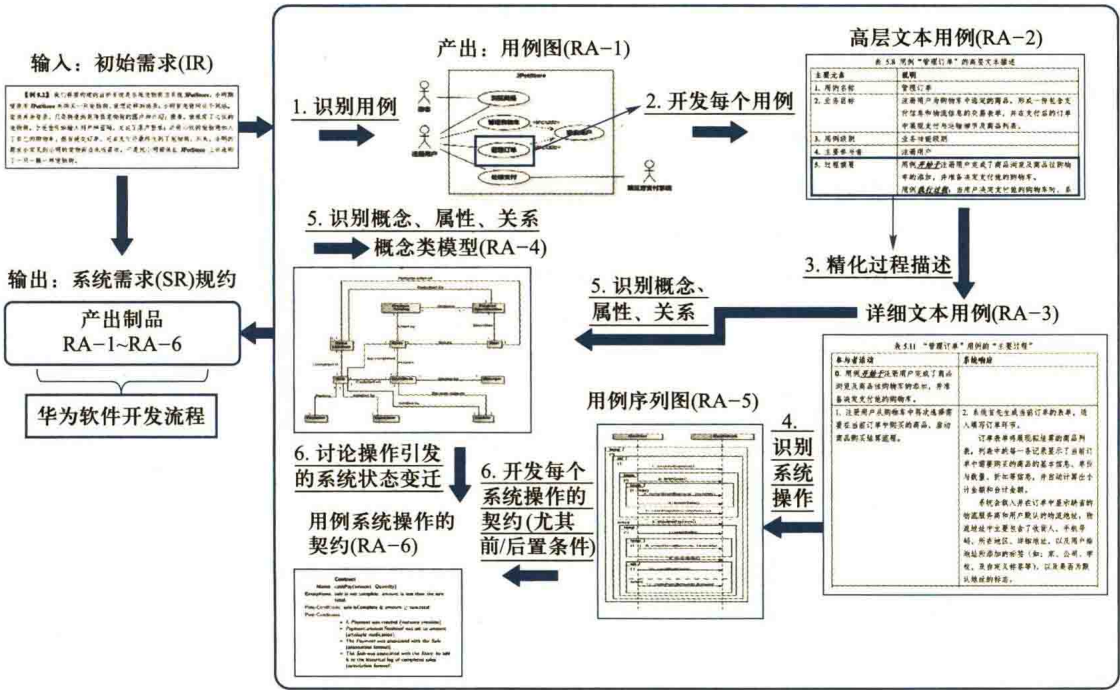
(overall textual use case), 在高层文本用例基础上,能对其实施增量式细化并持续识别新的子业务用例进而得到详细文本用例(detailed textual use case);③ 能够从详细文本用例等的描述中抽取有意义的概念,构造系统的概念类模型(conceptual class diagram),其中包括概念类的识别、类之间关系的定义(包括关系的类别、关系的属性和关系的性质定义)、概念类中有意义的性质的识别及其属性定义;④ 能够从详细文本用例中识别用例系统操作(use case system operation),并使用 UML 构造用例序列图(use case sequence diagram);⑤ 理解系统状态及系统状态变迁的实质,理解其霍尔三元组(Hoare triple)定义,能够针对用例序列图中的各个系统操作,运用霍尔三元组定义的思想来定义用例系统操作的契约(contract),并结合概念类图所定义的类和类间关系等约束条件,通过开发契约的前置条件(pre-condition)和后置条件(post-condition)来描述系统状态变迁。

(3) 了解需求分析阶段从事的开发活动。

完成需求分析阶段工作后,应能(依次)得到如下软件制品:

- (1) 用例图(制品编号^①:RA-1);
- (2) 高层文本用例(制品编号:RA-2);
- (3) 详细文本用例(制品编号:RA-3);
- (4) 概念类模型(制品编号:RA-4);
- (5) 用例序列图(制品编号:RA-5);
- (6) 用例系统操作的契约(制品编号:RA-6)。

【学习导图】



① 编号中的字母 RA 是 requirement analysis(需求分析)的缩写。

3.1 用例识别与开发

通过第二章的需求获取,读者已经得到用自然语言描述的用户需求描述。但这些需求描述未经系统化的整理和组织,可能是零散的、模糊的、不可实现的,或者某些描述之间存在不一致的情形。软件系统的设计人员无法直接基于这样的用户需求开展设计——尽管这是当前产业界许多中小企业事实上采用的软件生产方式,但本书强烈建议改进这样的软件生产方式和生产流程;而且,程序设计人员更无法根据这样的需求开始编码实现——这也是早期导致“软件危机”的根源之一。因此需要在理解既有需求描述的基础上,进行较为系统的需求整理和组织,得到结构化的需求描述,并就该需求描述同软件系统的需求方进行反复沟通和确认,以便为进一步构建以需求模型为核心制品的严格需求规约奠定基础。

本节将首先介绍用例的基本概念,继而解决其如何发现和识别问题,然后构造系统用例图(制品 RA-1)并完成用例的文本描述(制品 RA-2)。

3.1.1 用例的基本概念

关于用例的概念目前有多种定义,这里选取其中比较经典的两种。

Craig Larman 在 *Applying UML and Patterns* 一书^[22]中认为:

用例 (use case) 就是一组相关的成功和失败场景集合,用来描述参与者如何使用系统来实现其目标。

RUP 方法基于用例创立者 Ivar Jacobson^① 的思想^[23],提出:

(软件系统蕴含了)^②一组**用例实例 (use case instance)**,每个实例是系统所执行的一系列活动,以此产生对特定参与者具有价值的可观察结果。

从上述两个定义中,不难发现其共同点:

(1) 用例围绕参与者与系统的交互来展开,即特定参与者使用系统来完成其业务目标(获得有价值结果);

(2) 用例绝不是某个业务过程中的一个步骤(活动),而是系统所执行的“一系列”步骤(活动),以“完成目标”或获得“有价值结果”为要件。

基于上述分析,本章将用例定义如下:

用例是特定参与者为达成其具体的业务目标,通过与系统交互所执行的一系列完整的活动序列;该活动序列包括从业务开始、发展到结束期间的所有活动,并获得有业务价值的、稳定持久的结果;同时包含了成功和失败的情形。

对于该定义所蕴含的要点,本章作如下阐释:

(1) 用例必须是参与者与系统之间交互时的完整动作序列,而不是其中的某一步或某些步骤;

① Ivar Jacobson 同时是 UML 三大创始人之一,即 Grady Booch、James Rumbaugh 和 Ivar Jacobson。

② 该括号所包含的部分是编者根据对原作者思想的理解所增补的内容。

(2) 用例应围绕参与者所关心的具体业务目标,描述参与者与系统交互而产生有业务价值结果的全过程,而不是单方面描述系统能提供的功能特性;

(3) 用例通常情况下描述了正常情况下的完整动作序列(即业务执行成功的情形),但也不应忽视异常情况下的动作序列(即业务执行失败的情形)。

通过以上定义及阐释,读者应能较好地明确用例的含义,这将有助于接下来正确地识别用例。

3.1.2 用例的发现

用例是特定参与者为了达成具体的业务目标,使用系统执行的一系列完整的活动序列。软件开发项目将要构建的目标系统的业务需求,可以通过一组用例的实例(后文对用例和用例实例的概念不作区分,并统一称作“用例”)来描述。为确定目标系统包含哪些用例,需要完成两步操作:(1) **用例发现**:尽可能多地从参与者与系统之间的交互活动中,发现可作为候选的用例;(2) **用例识别**:根据用例的定义,从候选用例的集合中确定合适的用例。

(1) 用例发现的方法之一:参与者目标法

参与者目标法即明确参与者与系统之间的哪些交互活动能够产生对其“有业务价值的、稳定与持久的结果”。其具体执行步骤如下。

第一步:确定系统边界

需要明确项目的建设范围和总体目标,尤其需要注意正确地限定待开发系统需要实现的功能特性,排除那些与待开发系统发生交互的外部系统。

【案例 3.1】 以在线宠物商店系统 JPetStore 为例,读者需要判定用于处理支付业务的“支付系统”(如信用卡支付系统、微信支付系统、支付宝支付系统等)是否在待开发系统的边界之内。

读者可以分析来自第二章的需求文本。宠物采购订单在确认提交之后,需要完成支付行为,其支付行为由“处理订单支付”功能来响应,它是 JPetStore 系统必备的特性,理应在该系统的边界之内。接着,“处理订单支付”开始调用信用卡支付、微信支付、支付宝支付等各种支付渠道以完成具体的支付流程。一旦支付调用开始,JPetStore 系统内部的“处理订单支付”就将控制权(控制焦点)转移给具体某个(某些)支付渠道以完成支付处理流程,它只需要在线或离线地等待各支付系统处理完成并返回结果(支付成功或者支付失败)。显然,作为第三方支付系统的信用卡支付系统、微信支付系统、支付宝支付系统不应该在 JPetStore 系统的边界之内。

第二步:确定系统的直接参与者(直接用户及外部系统)

参与者既包括使用系统的用户(一般是用户角色,而不是具体某个用户),也包括与待开发系统之间发生交互的外部系统(例如具体完成支付行为的微信支付系统、支付宝支付系统等)。

首先讨论系统用户,其又可分为直接用户和间接用户。直接用户与系统之间发生直接的业务交互,间接用户通过直接用户同系统之间发生业务交互。确

定系统的主要参与者不需要包含间接用户。

确定直接参与者的方法之一,是通过分析直接使用待开发系统的具体用户角色方面来抽取,即首先找到直接使用的具体用户,而后对其角色进行抽象。

绘制一张“参与者-业务目标”表将有助于完成这项工作。该表设计为三列——具体用户、代表角色、业务目标,每列所代表的含义如表 3.1 所示。其中,“代表角色”一般情况下可以抽取为主要参与者。

表 3.1 “参与者-业务目标”表

具体用户	代表角色(参与者)	业务目标
(谁在实际使用待开发系统)	(使用待开发系统的用户属于什么角色)	(用户使用待开发系统的目的是什么)

【案例 3.2】 假设将要构建的待开发系统是在线宠物商店系统 JPetStore,小明期望使用 JPetStore 来购买一只宠物狗。设想如下场景:小明首先访问该网站,但他并未登录,只是快速浏览待售宠物狗的图片 and 介绍;接着他选定了心仪的宠物狗,于是开始输入用户名和密码,完成了账户登录;随后他将心仪的宠物狗加入了自己的购物车,然后提交订单、完成支付并最终收到了宠物狗。后来小明的朋友小宝见到小明的宠物狗后也很喜欢,于是委托小明帮他在 JPetStore 上也选购了一只一模一样的宠物狗。

在这个场景中,读者很容易确定具体用户是小明和小宝。但显然小明是使用 JPetStore 系统的直接用户,小宝则是通过小明使用 JPetStore 系统的间接用户。因此读者在填入“参与者-业务目标”表时,只需要关注作为直接用户的小明,而选择忽略间接用户小宝。同时需要特别注意的是,小明在使用目标系统的过程中使用过两种角色,即注册用户和非注册用户(游客)。在角色抽象过程中,不同的角色是需要进行区分的。因此,读者可以将“参与者-业务目标”表初步完善为表 3.2。

表 3.2 “参与者-业务目标”表(直接用户填入后)

具体用户	代表角色(参与者)	业务目标
小明	非注册用户(游客)	
小明	注册用户	

第三步:发现并构造候选用例集

在用例发现过程中,读者需要尽可能找出所有潜在的候选用例,这需要分析主要参与者与待开发系统之间所发生的业务交互活动,并构造候选用例集。这一阶段的主要任务并不是要直接确定“参与者-业务目标”表的业务目标,但可以将相应的交互活动均填入“参与者-业务目标”表的“业务目标”列,作为待开发系统的候选用例。当然,读者在每个候选用例后面可以用(?)进行标注,表明其是否适宜作为用例仍待后续环节的进一步识别。

从【案例 3.2】的场景描述中,读者可重点关注和分析动宾短语^①,不难提取出主要参与者“小明”与待开发系统 JPetStore 之间所发生的业务交互活动(如表 3.3 所示)。同时需要说明的是,虽然【案例 3.2】的场景描述并未特别指出小明作为注册用户是否能够实施“浏览商品”的行为,但根据人们的生活常识,应该容易判定这是完全有可能的、合理的。当然,这需要同软件系统需求方进行确认。

表 3.3 “参与者-业务目标”表(候选用例填入后)

具体用户	代表角色(参与者)	业务目标
小明	非注册用户(游客)	浏览商品(?)
小明	注册用户	浏览商品(?)
		输入用户名和密码(?)
		登录账户(?)
		管理购物车(?)
		管理订单(?)
		处理支付(?)

通过参与者目标法的上述三步操作,读者可以发现用例并得到待识别的候选用例集。

(2) 另一种用例发现的方法:事件分析法

需要注意的是,通过参与者目标法找到的参与者及候选用例集并不充分,尤其是它不太适用于发现与待开发系统发生交互的外部系统类型的参与者及与之相应的业务活动。此时需要引入另一种用例发现技术,即事件分析法。

所谓事件分析法,即分析系统发生的外部事件,明确其源点或终点,以确定参与者及与之相关的业务目标。读者可参考表 3.1 构造“外部事件”表。此处仍以【案例 3.1】的描述为例,可以分析得到“外部事件”表如表 3.4 所示。请读者注意:一些参与者前面被标注了【可选】,表明响应该外部事件的参与者既不唯一,也不要求全部都要响应。此外,在“业务目标”列标注(?)的含义与表 3.3 的含义一致。

表 3.4 “外部事件”表举例

外部事件	源点/终点参与者	业务目标
外部系统被请求完成待开发系统的订单支付	【可选】信用卡支付系统	(协助)处理支付(?)
	【可选】微信支付系统	
	【可选】支付宝支付系统	

① 有许多自然语言处理工具可帮助自动化处理这一过程,如 easyCRC、TextAnalysisOnline 等。

其他常见的外部事件包括：

- ① 外部系统调用待开发系统的服务；
- ② 外部系统在线监控待开发系统的性能；
- ③ 外部系统在线分析和处理待开发系统的日志或错误信息等。

3.1.3 用例的识别

本小节将对上一阶段获取的候选用例进行分析识别。在讲述识别用例的方法之前,请读者思考并回答以下问题:想象你去银行的自助服务区使用 ATM 系统办理业务的场景,你可能会执行:

活动①:输入密码

活动②:登录账户

活动③:取款/转账/查询

请问:你认为①~③的活动(候选用例)中,哪些(个)适宜作为用例,哪些(个)不适宜?你作出判定的依据是什么?

回顾 3.1.1 节关于用例的定义及其含义可知,作为一个用例,需要满足对下列要素的刻画,即:(1)完整动作序列;(2)具体业务目标;(3)成功与失败场景。据此思路分析如下。

① 输入密码

- 它代表了某个业务的完整动作序列吗?大多数情况下并非如此。它只是某个完整动作序列中的某一个步骤。

- 它代表了特定参与者想要实现的具体业务目标吗?同样并非如此。问问自己:今天我去银行自助区使用 ATM 系统的目的就是输入密码?这显然不合理。^①

- 它涵盖了成功与失败场景吗?或许如此。例如:用户发现自己多输入了一个字母并立即更正,这包含失败场景及其处理。如果用户多输入了一个字母但并未发现,此时“输入密码”活动便不包含对失败场景的处理;根据人们的生活经验,在这种情况下,需要“输入密码”活动之外的“密码校验”活动来处理失败场景。

② 登录账户

- 它代表了某个业务的完整动作序列吗?或许如此。在许多情况下,登录账户只是完成某个完整动作序列(如取款流程)的一个步骤。但也不能完全否认,登录账户也可能包含了完整动作序列,例如,它开始于打开登录界面,结束于登录成功,中间流程包括输入账号、输入密码、提示验证码、发送验证码、输入验证码等。从这个角度理解,登录账户是可以蕴含完整流程的。只是相对于取款、转账、查询等活动而言,它们不处于同一个抽象层次(粒度)。

- 它代表了特定参与者想要实现的具体业务目标吗?想象一下,你家里那

^① 这种方式也被称作“老板测试”。

位勤劳的、忙碌一上午的妻子(或母亲)没见到你,于是质问你`你在干什么`,你回答:今天我花了一上午时间去银行自助区使用 ATM 系统登录账户去了。这个回答大概很难被认为是“合理”的。

● 它涵盖了成功与失败场景吗?这当然是完全有可能的。失败场景包括:密码错误时拒绝登录,并提示用户是否需要进入找回密码的流程,无该用户名时提示用户是否需要进入注册新用户的流程等。想象你回答“今天我花了一上午时间去银行自助区使用 ATM 系统登录账户去了”之后开始的“狡辩”:我之所以花一上午登录账户,是因为登录时总是不成功,最后还被吞掉银行卡;于是不得不求助银行柜台的工作人员,要求他们走异常处理流程帮我取回卡片,所以才耽误了一上午(显然此时“合理性”大大提升)。

③ 取款/转账/查询

● 它代表了某个业务的完整动作序列吗?是的。无论取款、转账还是查询,均包含了完整的流程。该流程开始于用户身份验证(插卡、输入密码、登录账户),完结于业务处理成功(以取款为例:取款成功、拿走钞票、取出卡片、离开银行自助区),中间包含了业务处理的中间环节(以取款为例:输入取款金额、检验账户余额、扣款吐钞、余额扣减等)。

● 它代表了特定参与者想要实现的具体业务目标吗?是的。今天上午我去银行自助区使用 ATM 系统的目的是取款,这是缺席上午家庭事务的非常合理的解释。

● 它涵盖了成功与失败场景吗?当然。取款流程动作序列中,许多环节都存在发生失败的可能性,都有相应的失败响应流程。

综上,输入密码显然不适宜作为用例,取款/转账/查询显然应当作为用例。而登录账户则存在争议,但它显然不适宜作为取款/转账/查询同一级别的用例。

Craig Larman 在其 *Applying UML and Patterns* 一书中提出,允许“老板测试”中合理的违例,即作为子功能(子业务)级别的候选用例,如满足以下情形,可以考虑将其作为单独的用例。

允许违例情形:当子功能(子业务)级别的候选用例被多个用例所使用,即使它不能够通过“老板测试”,为了避免内容重复,也可将其单独作为用例,通过扩展或包含的形式连接到其他用例上。

显然,存在争议的候选用例“登录账户”则属于此类情形。取款时需要先行登录,查询时和转账时都需要使用“登录账户”的子功能(子业务)。

通过上述分析以及对违例情形的讨论,读者应能对 3.1.2 节所获得的候选用例集完成用例识别并确定用例,如表 3.5 和表 3.6 所示。

表 3.5 “参与者-业务目标”表(用例识别后)

具体用户	代表角色(参与者)	业务目标
小明	非注册用户(游客)	浏览商品(√)

续表

具体用户	代表角色(参与者)	业务目标
小明	注册用户	浏览商品(√)
		输入用户名和密码(×)
		登录账户(√-违例条款)
		管理购物车(√)
		管理订单(√)
		处理支付(√)

表 3.6 “外部事件”表(用例识别后)

外部事件	源点/终点参与者	业务目标
外部系统被请求完成待开发系统的订单支付	【可选】信用卡支付系统	(协助)处理支付(√)
	【可选】微信支付系统	
	【可选】支付宝支付系统	

读者可反复追踪分析 JPetStore 可能的业务使用场景,按照上述两节(3.1.2 节和 3.1.3 节)关于用例发现和识别的流程,完成对整个 JPetStore 系统的用例提取。

3.1.4 开发用例图(RA-1)

通过 3.1.3 节完成了用例的发现和识别,并最终得到 JPetStore 在用户在线购物场景下的用例集合,相关结果见表 3.5 和表 3.6。接下来,本小节将根据用例识别结果开发用例图(RA-1)。用例图通过刻画参与者、用例以及它们之间的关系(包括参与者与参与者之间的关系、用例与用例之间的关系,以及参与者与用例之间的关系)来展现系统的功能。

UML 为用例图的各元素提供了相应的图形表示,常用的用例图元素的图形表示包括系统边界、参与者、用例、关系四类。以图 3.1 为例进行介绍,而图 3.1 事实上就是基于表 3.5 和表 3.6 的用例识别结果所开发的用例图。

(1) **系统边界**:图 3.1 中命名为 JPetStore 的方框代表了 JPetStore 系统的边界。系统边界展现了系统运行的上下文(context):位于边界之内的部分代表了系统本身所能提供的功能特性,所有的用例都应该位于相应的系统边界之内;位于边界之外的部分代表了使用系统或与系统发送交互的参与者。

(2) **参与者**:图 3.1 中的游客、注册用户、第三方支付系统(即信用卡、支付宝、微信的支付系统)均是参与者,其元素表示类似“火柴人”。读者可以注意到,三个参与者中,游客和注册用户位于 JPetStore 系统边界的左侧,而第三方支付系统位于 JPetStore 系统边界的右侧。



图 3.1 用例图(示例)①

关于不同参与者的这种左右布局并非随意确定的。前文提到,参与者可分为直接参与者和间接参与者,而用例图仅关注直接参与者;而直接参与者又可再分为**主要参与者**(primary actor)和**协助参与者**(supporting actor)。所谓主要参与者,即主动触发用例、驱动用例执行完整活动序列并实现该参与者期望的业务目标的参与者;协助参与者指的是参与用例执行过程,并为用例提供服务以协助其完成相关活动序列,实现主要参与者业务目标的参与者。在绘制参与者时,一般将主要参与者(游客和注册用户)绘制在系统边界的左侧,将协助参与者(第三方支付系统)绘制在系统边界的右侧。

(3) 用例:标准 UML 图例中,常使用椭圆来表示用例,根据表 3.5 和表 3.6 的用例识别结果,可以绘制图 3.1 中的用例,包括浏览商品、管理购物车、管理订单、处理支付,以及在违例条款下许可的用例登录账户。需要特别提醒的是,用例之间不存在流程上的先后序关系,用例内部才包含完成某个业务的活动序列^②,用例之间在业务上是相对独立的,这也体现了“高内聚低耦合”的思想。一些读者通常将用例图画成了由若干用例按先后顺序或某些业务上的逻辑关系构成的“流程图”,这是不正确的用例图构建结果。

(4) 关系:用例图中主要刻画三类关系:参与者与用例之间的关联关系(用直线表示),参与者之间的关系(泛化关系,用直线+三角箭头表示,箭头指向父类型),用例与用例之间的关系(包含或扩展关系,用虚线+箭头+<<include>>或<<extend>>标注表示)。各关系的 UML 图示见图 3.2。

● 关联关系:以图 3.1 为例,游客与浏览商品之间即参与者与用例之间的关联关系,第三方支付系统与处理支付之间也是参与者与用例之间的关联关系。

① 使用授权软件 StarUML v4.0.1 绘制。

② 初学者往往将用例图绘制成一张流程图,这是把用例内部的活动序列和用例之间的关系混淆了。用例之间主要是包含和扩展关系,而非流程控制上的先后序关系。

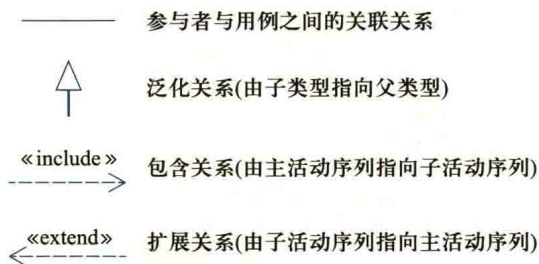


图 3.2 用例图中常用的关系表达

● 泛化关系:为了更好地表达用例所运行的上下文,参与者与参与者之间的关系也可以表达出来。比如,游客和注册用户可以泛化一个参与者“系统用户”,即参与者游客和注册用户可以继承高层概念上的参与者“系统用户”——这种做法的实践意义在于:当初始需求分析时,可以先描述高层概念上的参与者;在后续分析环节中,再通过继承的方式来细化。

除了参与者与用例之间的关系外,用例之间的关系也需要展现在用例图中。用例之间的关系主要有以下两类^①。

● 包含关系:表示为虚线箭头并在虚线上标注<<include>>,箭头的方向是从基本用例指向被包含的用例。包含关系表示某个用例在其活动序列中包含了另一个用例的活动序列,例如:在管理购物车用例中,如果要将某商品加入个人的购物车,则要求当前用户必须为注册用户,因此,在执行管理购物车用例时需要首先完成登录账户的动作序列。

● 扩展关系:表示为虚线箭头并在虚线上标注<<extend>>,箭头方向是从扩展用例指向基本用例(注意和包含关系中的箭头方向恰好相反)。扩展关系表示将一些常规的动作序列放在基本用例中,将可选的或只在特定条件下才执行的动作序列放在其扩展用例中。例如:在管理订单用例中,包含了选择购物车商品、形成采购订单、计算支付金额、提交订单等常规动作序列;在某些情况下,例如用户不确定所采购的商品是否真的是自己期望的物品,需要在拿到商品并试用后再决定是否保留;此时用户可以选择购买运费险^②,来避免可能的退货损失。因此,购买运费险用例就成了管理订单用例的扩展用例。虚线箭头由购买运费险用例指向管理订单用例,并在虚线上标注<<extend>>,如图 3.3 所示(注意图中标有*的“购买运费险”用例并非根据前文用例识别的结果得到,而是为了解释此处的扩展关系而额外添加的)。

当然,此时读者得到的如图 3.1 或图 3.3 的用例图,还没有覆盖 JPetStore 系统中的所有用例。读者可以按照 3.1.3 节的方法来发现和识别其他用例,并按照

^① 当然,用例之间还可以存在其他关系(如继承关系)。但 Flower 和 Cockburn 等世界级用例专家均认为:需求分析阶段更重要的工作并非用例图,而是接下来将要讨论的用例文本。受限于篇幅,这里不再赘述更多用例图开发的细节,感兴趣的读者可查阅 UML 规范。

^② 可参考淘宝的购物流程,在购买了运费险的情况下,如果采购的商品不满意,在完成退货后,退货所产生的物流费用可由运费险的承保人赔付。

本节的用例表示法,将图 3.1 或图 3.3 的用例图补充完整。

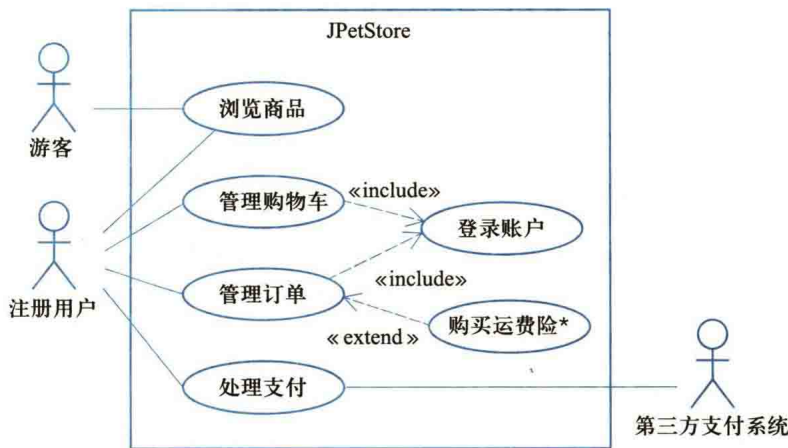


图 3.3 用例间的包含与扩展关系示例

通俗地说,通过用例图,软件系统的开发团队能够得知系统都有哪些功能特性(一个用例一般代表一个功能特性);但是,读者也应明确,通过用例图,并不能得知用例内部的动作序列如何发生——这些应该是接下来两节介绍的“文本用例开发”需要解决的问题。

3.1.5 高层文本用例开发(RA-2)

如前所述,通过用例图(RA-1),只能知道系统的主要功能有哪些,但是无法得知用例内部的动作序列,而动作序列恰恰蕴含了用例具体的业务过程。因此,相对于用例图而言,针对各用例的文本描述(RA-2 和 RA-3)提供了更多的需求细节。

虽然对用例进行的文本描述(RA-2 和 RA-3)是使用人们日常所用的自然语言来完成的,但这并不意味着这种自然语言的文本描述是随意进行的;相反,它通常是结构化的,结构化的文本描述有助于更好地组织和展现用例文本。在软件工程的实践过程中,使用最广泛的用例文本的结构模板是 20 世纪 90 年代由 Alistair Cockburn 所创建的模板:

```
Use Case: <number> <the name should be the goal as a short active verb phrase>
Goal in Context: <a longer statement of the goal, if needed>
Scope: <what system is being considered black-box under design>
Level: <one of: Summary, Primary task, Subfunction>
Primary Actor: <a role name for the primary actor, or description>
Priority: <how critical to your system / organization>
Frequency: <how often it is expected to happen>
```

本节参考了该模板,并根据高层描述的需要进行了裁剪和增补,本书采用的

模板如表 3.7 所示。

表 3.7 用例高层文本描述模板

主要元素	说明
1. 用例名称	用例名称与先前识别的用例名称一致
2. 业务目标	主要参与者执行本用例期望达到的业务目标
3. 用例级别	业务功能级别(如管理购物车)或子业务(子功能)级别(如登录账户)。在高层文本描述中,一般只涉及业务功能级别用例,在进行细化之后才会抽取出子业务(子功能)级别用例
4. 主要参与者	主动触发用例、驱动用例执行完整活动序列并实现该参与者期望的业务目标的参与者
5. 过程摘要	此处简要描述用例成功执行的基本过程,通常用于快速地了解当前用例的主题和范围。根据识别用例的方法,此处需要表达出用例作为一个“完整的动作序列”应具有的特征,即需要描述用例开始和结束的行为,以及发生在始末之间的过程(尽管在高层描述时过程并不会太详尽)。本书建议显式地使用“开始于”“结束于”“执行过程”等关键字来辅助描述用例执行时的过程摘要

【案例 3.3】 如表 3.8 所示,以“管理订单”的用例为例,可将其高层文本描述开发如下:

表 3.8 用例“管理订单”的高层文本描述

主要元素	说明
1. 用例名称	管理订单
2. 业务目标	注册用户为购物车中选定的商品形成一份包含支付信息和物流信息的交易表单,并在支付后的订单中展现支付与运输细节及商品列表
3. 用例级别	业务功能级别
4. 主要参与者	注册用户
5. 过程摘要	用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付 用例执行过程:当用户决定支付时,系统会构造一个支付表单,表单包含待购商品列表及价格,并要求注册用户选择支付途径、提供支付信息并填写物流地址;付款完成时系统会创建一个订单,其中包括支付与运输细节及商品列表 用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看

高层文本描述适用于需求分析阶段的早期,能快速地针对用例识别后的用

例集合进行各用例内部执行序列的概括描述(该做法也可用于敏捷开发方法)。

3.1.6 详细文本用例开发(RA-3)

获得用例图中各个用例的高层文本描述后,读者就有了通过进一步细化来开发需求分析阶段的(最)重要产出物——详细文本用例(RA-3)的基础。

用例高层文本描述的增量式细化通常发生在需求分析阶段的中期,此时已经确定并以摘要形式编写了大量用例。一般而言,软件项目的需求分析团队通常会选择所有用例中最具影响力(或最关键业务)的一部分用例^①率先实施这一细化工作,后续再根据项目需要对剩余的用例进行迭代。

跟用例的高层描述一样,通常也是通过结构化的描述模板来完成用例描述的细化。一般而言,可以采用如表 3.9 所示的结构化描述模板。其中最主要的工作是对高层描述中的“过程摘要”进行扩展,其扩展内容将构成前置条件、后置条件(又称为成功保证)、主要过程、扩展事件等刻画详细文本用例的核心要素。

表 3.9 用例详细文本描述模板

主要元素	说明
1. 用例名称	用例名称与先前识别的用例名称一致(该部分可直接沿用表 3.7 的内容)
2. 业务目标	主要参与者执行本用例期望达到的业务目标(该部分可直接沿用表 3.7 的内容)
3. 用例级别	业务功能级别(如管理购物车)或子业务(子功能)级别(如登录账户)。在高层文本描述中,一般只涉及业务功能级别用例,在进行细化之后才会抽取出子业务(子功能)级别用例(该部分可直接沿用表 3.7 的内容)
4. 主要参与者	主动触发用例、驱动用例执行完整活动序列并实现该参与者期望的业务目标的参与者(该部分可直接沿用表 3.7 的内容)
5. 过程摘要	此处描述的是用例成功执行的过程,这是用例文本描述的核心。根据识别用例的方法,此处需要表达出用例作为一个“完整的动作序列”应具有的特征,即需要描述用例开始和结束的行为,以及发生在始末之间的简要过程。本书建议显式地使用“开始于”“结束于”“执行过程”等关键字来辅助描述用例执行时的过程摘要(该部分可直接沿用表 3.7 的内容)
6. 前置条件	交代用例在开始执行之前必须满足的条件。例:可从表 3.8 中“5. 过程摘要”的“开始于……”描述部分得到启发

^① Craig Larman 在其 *Applying UML and Patterns*(3rd ed)一书中推荐先对 10%具有重要架构意义和高价值的用例进行详细文本用例的开发,以及实施后续的设计和编程。

续表

主要元素	说明
7. 后置条件	描述用例在执行完毕之后需要得到的成功保证。例：可从表 3.8 中“5. 过程摘要”的“结束于……”描述部分得到启发
8. 主要过程	本部分实际是对“5. 过程摘要”的细化,主要描述典型的、理想状态下用例成功执行的场景。其表述过程主要考虑“主要参与者”与“系统”两个对象之间的交互,即考虑主要参与者的请求序列及系统对相关请求的响应
9. 扩展事件	本部分是对“8. 主要过程”的补充,主要描述用例执行过程中异常事件发生时的处理过程(如支付时发生余额不足时的处理场景),或主要场景中针对某些活动序列的可替换(分支)序列(如将现金支付的活动序列替换为支付宝支付的活动序列)
10. 非功能性需求	描述用户对响应时间、系统接口、服务质量等方面的需求
11. 其他	可描述用例执行频率等能够辅助设计决策的信息

根据上述模板,本节基于【案例 3.3】继续对“管理订单”的文本用例进行开发。

【案例 3.4】“管理订单”的详细文本用例开发步骤如下。

第一步:沿用高层文本用例的内容来填充详细文本用例的模板

这一步可先得到表 3.10 的部分文本用例。

表 3.10 使用高层文本用例信息填充后的详细文本用例

主要元素	说明
1. 用例名称	管理订单
2. 业务目标	注册用户为购物车中选定的商品形成一份包含支付信息和物流信息的交易表单,并在支付后的订单中展现支付与运输细节及商品列表
3. 用例级别	业务功能级别
4. 主要参与者	注册用户
5. 过程摘要	用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付 用例执行过程:当用户决定支付时,系统会构造一个支付表单,表单包含待购商品列表及价格,并要求注册用户选择支付途径、提供支付信息并填写物流地址;付款完成时系统会创建一个订单,其中包括支付与运输细节及商品列表 用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看
6. 前置条件	TODO:在接下来的第二步分析步骤中完成
7. 后置条件	TODO:在接下来的第二步分析步骤中完成
8. 主要过程	TODO:在接下来的第三步分析步骤中完成

续表

主要元素	说明
9. 扩展事件	TODO:在接下来的第四步分析步骤中完成
10. 非功能性需求	TODO:在接下来的第五步分析步骤中完成
11. 其他	TODO:在接下来的第五步分析步骤中完成

第二步:分析并开发“前置条件”与“后置条件”

首先分析“过程摘要”中的“开始于……”描述部分:“用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付”。读者可以从“哪些对象被创建或销毁、哪些对象的属性会被修改或发生变更、哪些对象之间的连接关系会被建立或销毁”三个思考角度^①对前置条件进行描述:

- 用户的购物车(对象)应存在;
- 所选购的商品(对象)应存在;
- 所选购的商品(对象)应关联到购物车(对象)(多对一),代表商品向购物车的添加;
- 购物车(对象)应关联到当前用户(对象)(一对一),代表该购物车属当前注册用户所专有。

接着分析“过程摘要”中的“结束于……”描述部分:“用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看。”同样地,读者可以从“对象创建或销毁、对象属性变更、对象间连接关系创建或销毁”三个角度对后置条件进行如下描述:

- 订单(对象)应存在;
- 订单(对象)的支付状态应设置为“已支付”,订单(对象)的支付信息应更新为实际支付途径所包含的信息(如支付宝、微信的支付信息和支付回执);
- 订单(对象)的物流状态应更新为“待发货”(或其他第三方物流服务所定义的物流状态),订单(对象)的物流信息应更新为实际物流服务所包含的信息(如顺丰物流、申通物流等第三方物流服务系统所提供的物流信息);
- 所交易的商品(对象)应存在,并关联到当前的订单(对象)(多对一),代表商品交易真实发生,且所交易的商品恰好在当前订单所对应的商品列表中,同时根据商品列表能追踪到各个交易商品的信息,不会链接到空的商品(对象)。

需要注意的是,前置条件和后置条件的描述将有助于测试用例的设计和实现。特别地,后置条件将成为后续的系统设计环节中,构造对象序列图时进行对象职责分配的设计依据(读者可参阅第五章相关内容)。

第三步:分析并完成“主要过程”描述

“主要过程”是用例文本描述中最重要的部分,也是后续环节识别和开发“用

^① 至于为什么会从这三个角度去理解业务过程或者系统的行为,读者可参考“3.4.1 系统状态变迁的本质”一节中的讨论。

例系统操作(use case system operation) ”的依据。它描述典型的、理想状态下用例成功执行的场景,主要刻画当前用例背景下主要参与者的请求序列及系统对相关请求的响应。

一般而言,场景中的三种活动会被记录在“主要过程”的描述中:

- (1) 参与者之间的交互;
- (2) 参与者的请求与系统的响应;
- (3) 系统内部发生的状态变化(特别提示:“系统状态变化”所蕴含的意义是对象创建或销毁、对象属性变更、对象间连接关系创建或销毁,可参考阅读“3.4.1 系统状态变迁的本质”)。

接下来开始分析“过程描述”中的“用例执行过程”描述部分:“当用户决定支付时,系统会构造一个支付表单,表单包含待购商品列表及价格,并要求注册用户选择支付途径、提供支付信息并填写物流地址;付款完成时系统会创建一个订单,其中包括支付与运输细节及商品列表。”显然,针对这部分内容的高度概括,需要根据需求获取环节所得到的需求信息,结合业务专家的领域知识以及需求分析师自身在软件系统开发方面的经验,将相关细节补充完整。

为了更好地表达主要过程中主要参与者与系统之间的交互行为,通常采取一种称为“双栏对话式”的表示法^①,该表示法将用例执行过程表示为“参与者活动”与“系统响应”之间的交互。

用例“管理订单”的“主要过程”开发如表 3.11 所示。将触发“主要流程”开始执行的那部分活动(即“开始于……”描述部分)作为主要参与者在“主要流程”中的初始活动(编号为 0),以便启动本用例。将标志当前用例结束的活动(即“结束于……”描述部分)作为“主要流程”的最后一个活动。同时,对参与者与系统之间的每次请求或响应均按顺序编号,以便在后续的“扩展事件”描述中加以引用。

表 3.11 “管理订单”用例的“主要过程”

参与者活动	系统响应(What/How)
0. 用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付	
1. 注册用户从购物车中再次选择需要在当前订单中购买的商品,启动商品购买结算流程	2. 系统首先生成当前订单的表单,进入填写订单环节。订单表单将展现拟结算的商品列表,列表中的每一条记录显示了当前订单中需要购买的商品的基本信息、单价与数量、折扣等信息,并自动计算出小计金额和合计金额。系统会载入并在订单中显示缺省的物流服务商和用户默认的物流地址,物流地址中主要包含收货人、手机号码、所在地区、详细地址,以及用户给地址添加的标签(如家、公司、学校及自定义标签等)和是否为默认地址的标志

^① 该表示法由 Rebecca J. Wirfs-Brock 在 *Designing Scenarios: Making the Case for a Use Case Framework* 中首次提出,并在软件工程领域广泛使用。另一种为“单栏式”表示法,其特点是简单紧凑、易于格式化。但相较而言,“双栏对话式”表示法的用例文本表达更加清楚和形象,易于平滑地转换为系统序列图。

续表

参与者活动	系统响应 (What/How)
3. 当前用户确认物流服务商的选择,并 自己预留的物流地址列表选择一个地址, 作为当前订单中的商品期望送达的地址	4. 系统根据当前用户对物流服务商及物 流地址的选择确认,自动更新订单中的运费 数额,并再次计算得到更新后的合计金额, 且将订单状态置为“待支付”。同时系统会 载入并提示当前系统支持的可选支付方式, 还会提示当前用户录入并确认发票信息(默 认不开发票)
5. 当前用户选择确认支付方式,并确认发 票信息,发票信息包含发票类型、发票抬头、 收票人信息、发票内容等。随后用户选择提 交支付	6. 系统根据当前用户的支付选择转入支 付流程,并进入“处理支付”用例。在支付成 功后,返回本用例,并将订单状态置为“已支 付”。在订单完成支付后,系统将向各商品 的销售商告知支付状态并通知备货发货
7. 各商品的销售商在获得系统通知后,将 按照订单物流要求,通过订单选定的物流服 务商发运商品,并获得各自商品的物流运单 号提交给系统	8. 系统在收到当前订单对应各商品的运 单号后,将各商品同运单号在系统中关联起 来。同时系统会根据运单号请求物流服务 商返回并载入本订单所对应各商品的相关 物流运输细节(包括订单号、商品标识符、物 流运单号、物流运输状态,以及在电子地图 中实时展现的可视化物流运输的详情)。当 前用户此时可查看订单详情,包括商品列 表、物流信息、支付信息、发票信息
9. 用例结束于注册用户在支付完成后对 订单中有关支付与运输细节及商品列表的 查看	

当然,限于篇幅,相较于真正软件项目中的用例“主要过程”描述,表 3.11 仍然是粗略的。在真实项目中,根据需要对其进行进一步细化也是很有必要的。

第四步:分析并完成“扩展事件”描述

扩展事件是对主要过程的补充,描述用例执行过程中异常事件发生时的处理过程(失败情况下的流程)或主要场景中针对某些活动序列的可替换序列(分支流程)。在理想状态下,主要流程与扩展事件描述的结合,应能够覆盖所有利益相关者在当前用例中所关注的业务目标。

扩展事件的描述包括三个部分:编号、触发条件以及处理过程。

(1) 编号:编号部分应与“主要流程”的活动序列中事件发生所处的活动位置有关。例如:在表 3.11 的示例中,如果当前用户本次订单采购的目的是给远方的朋友网购一份节日礼物,而朋友的地址并非当前用户预留的常用地址,此时则需要新增一个物流地址。而新增物流地址的事件应该发生在表 3.11 的活动 3,因此,“新增物流地址”的扩展事件应该编号为“3a”。若针对活动 3 有多个扩展事件,可依次编号为 3a、3b、3c……

如果某些扩展事件在多个活动中发生,例如“当前用户登录已超时”在活动2、3、4、5中发生,则可编号为2-5a。若针对上述活动有多个扩展事件,可依次编号为2-5a、2-5b、2-5c……

如果某些扩展事件在绝大多数活动中都可能发生,例如用户不待操作完成直接终止了操作,则可编号为*a。若有多个类似事件,则可依次编号为*a、*b、*c……

(2) 触发条件:触发条件用于表达当前扩展事件在什么样的情况下才会发生。例如:在表3.11的示例中,活动6可能会发生“支付失败”的情形,此时收到由“处理支付”用例返回的“支付失败”的消息就是一种“处理支付失败事件”的触发条件。此外需要注意的是,触发条件应尽可能使用可检测的描述作为条件,例如:

- “收到由‘处理支付’用例返回的‘支付失败’的消息”;
- 支付出现了问题。

针对以上两种事件触发条件的描述,显然前一个描述可检测性更好,更适宜作为扩展事件的触发条件。

(3) 处理过程:处理过程可以是一个活动,也可以是一个活动序列,甚至可以直接引用其他用例蕴含的完整动作序列来完成。扩展事件的处理完成后,系统默认重新回到“主要过程”的流程中。

关于扩展事件的描述,此处以表3.12为例进行示范。

表 3.12 “扩展事件”描述举例

描述项	具体内容
编号	3a(表示发生在活动3的第一个扩展事件)
触发条件	当前订单所购商品期望送达的地址不在系统可选的物流地址列表中
处理过程	1. 选择新增物流地址 2. 在新增物流地址的表单中,填入收货人、手机号码、所在地区、详细地址,以及用户给地址添加的标签(如家、公司、学校及自定义标签等)和是否为默认地址的标志等信息后,确认并提交 3. 此时系统载入新填入的物流地址,并返回执行本用例的主流程

第五步:分析并完成“非功能性需求”及“其他”部分的描述

“非功能性需求”部分用于描述与用例相关的软件质量特性和设计约束。相关软件质量特性可以参考ISO/IEC 25010:2011 *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models* 或GB/T 25000.10《系统与软件工程 系统与软件质量要求和评价(SQuaRE) 第10部分:系统与软件质量模型》两份标准中的定义。

这些质量特性包括产品质量特性(如性能效率、兼容性、易用性、可靠性、信息安全、维护性和可移植性等)和使用质量特性(如有效性、效率、满意度、抗风

险性等)两大类。各产品质量特性均包含子特性,如性能效率质量特性所蕴含的时间特性、资源利用性等子特性,以及可靠性质量特性所蕴含的容错性、易恢复性等子特性。使用质量特性也包含相关子特性,如满意度所蕴含的愉悦性、舒适性等子特性。

- 非功能性需求描述的案例如下:
- 当使用触屏显示器使用本系统时,文本信息的可见距离为 0.5 米;
 - 当提交支付时,系统相应的时间应该小于 30 秒。
- “其他”部分用于描述一些技术变元,这些技术变元通常涉及系统的利益相关者期望的特殊设计约束,例如要求订单支付时只能使用电子数字签名技术。
- 至此,“管理订单”的详细文本用例开发完毕。

3.2 系统结构建模 (RA-4)

在面向对象的软件工程方法中,系统架构的静态结构一般用类图来表达。本节首先介绍类图相关的术语,然后讨论概念类模型 (RA-4) 的开发过程,即概念与属性的识别、类间关系的定义。

3.2.1 类图相关的术语

首先明确面向对象方法的一些术语。

(1) 类 (class):在面向对象方法中,类表示具有相同特性(数据元素)和行为(功能)的对象的抽象。类可以用 UML 类图(class diagram)来表示。类图包含三部分(如图 3.4 所示),即类名、属性(数据元素)、方法(行为/功能)。

根据日常生产生活经验,若要描述某个领域的某些业务,必须首先明确该领域所涉及的一些重要概念 (concept),在此基础上方能描述相应的业务活动。例如,若要描述物流领域的一个仓库管理业务,就需要明确“仓库”“订单”“货架”等概念;若要描述网商领域的一个在线购物支付业务,就需要明确“收藏夹”“购物车”“快递单”“运费险”等概念。可见概念在刻画相关领域模型时的重要性。

在需求分析阶段,这些概念使用类和类图予以抽象和表达;且在抽象和表达时可暂时忽略其行为部分,只关心其概念(类名)、重要的数据信息(属性)以及概念和概念之间的联系(关系),因而称这样的类为概念类 (conceptual class),这样的类图为概念类图 (conceptual class diagram),如图 3.5 所示。当然,概念类的更为正式的定义是由其符号(名字或标识)、内涵(描述一个事物区别于其他事物的本质特性)和外延(列举该概念所适用的示例)来刻画的。

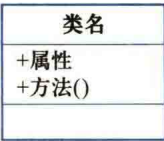


图 3.4 UML 类图

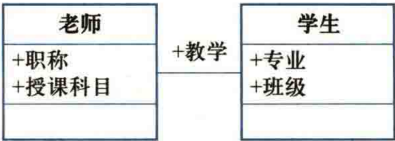


图 3.5 概念类图举例

(2) 对象(object):对象指相关业务活动所涉及的具体一个事物,它是类的实例。事实上,通常是从具体对象的业务活动中抽象出类及类之间的关系。例如,可以从图 3.6 的对象图抽象出图 3.5 的类图。



图 3.6 对象图举例

从图 3.6 的对象图中,读者也可以看到“老师”的前面标记了“张三”,表示“老师”类下的一个实例(对象)“张三”;对象“张三”的属性“职称”取值为“副教授”,属性“授课科目”取值为“系分”。业务活动的实际执行都是通过具体的实例(对象)而非概念(类)来进行的。

(3) 关系(association):刻画类与类之间的联系(如图 3.5 中的“教学”)。关于关系的类别和性质,将在 3.2.3 节予以具体描述。

(4) 连接(link):刻画对象和对象之间的关系(如图 3.6 中的“教学”)。类比对对象和类之间的联系,连接是关系的实例,因此图 3.5 中的“教学”关系和图 3.6 中的“教学”连接不是一回事。例如:图 3.5 中的“教学”关系可以是一个“多对多”的关系(n vs. n),表示一个教师可以给多个学生实施教学,而一个学生可以选择由多个教师讲授的课程;而图 3.6 中的“教学”连接就是具体的关系描述了,这个具体的连接可以是:张三老师可以给“大二(1)班”60 名同学上“系分”课(其中包含李四同学),而大二(1)班的“系分”课可能只由“张三”一名老师来上(张三老师同时上理论课和实验课),他们之间构成了“一对多”的连接(1 vs. 60)——而“一对多”的连接是满足教师与学生之间的“多对多”关系约束(n vs. n)的(因为 n 可以取 $1, 2, 3 \dots 60, 61, 62 \dots$)。

3.2.2 概念及重要属性识别

捕获概念以构造概念类及其属性的候选集合,继而予以甄选识别,过程如下。

(1) 发现概念与属性。研究目标领域的业务背景、工作环境、知识与术语体系等进行发现。研读用户方提供的各种文档、现有系统及实地观察所捕获的用户需求描述,以及系统开发方在前期构造的详细文本用例,围绕待开发系统构建的需要,选取其中相关的名词加入候选概念(类)和属性集合。从系统行为建模时识别的系统操作中,选取系统操作所指向的对象加入候选概念(类)和属性集合。

例如,开发一个教室管理系统,可以根据教学工作环境,考虑将教室教学活动相关的业务对象(讲台、课桌、座椅、投影仪、日光灯、空调等)、场所(教室、值班室等)、角色(教师、学生等)作为概念。

(2) 甄选概念与属性。一般而言,在完成相关业务时,需要通过系统进行操作和控制的对象或属性,才需要将其纳入概念类及其属性的集合。

此处仍以教室管理系统为例:如果需要通过系统来管理教室内的座椅,则需要一个“Chair”类;如果只需要对座椅进行简单管控——即可用还是不可用,则需要“Chair”类有一个类型为 boolean 的“available”属性;当然,如果考虑到“为学生提供最好的教学服务”这一目标,让学生们坐着学习时更舒适,则需要座椅能够调节靠背角度、座椅温度等,此时需要“Chair”类除了拥有“boolean available”属性之外,还需要拥有“int angle”“int temperature”等属性,以便实现座椅的角度、温度可操控。

(3) 开发初始概念类图。初始概念类图由一组没有定义操作的类图构成,其目的是展现某个业务领域的对象或概念,以及相关对象或概念的重要属性。需要将第二步得到的概念和属性以 UML 类图的形式表达出来(如图 3.7 所示)。之所以添加修饰词“初始”,是因为至此仍然没有完成对概念之间关系的刻画——这部分内容将在下一小节进行讨论,完成类间关系定义之后方可得到最终的概念类图。

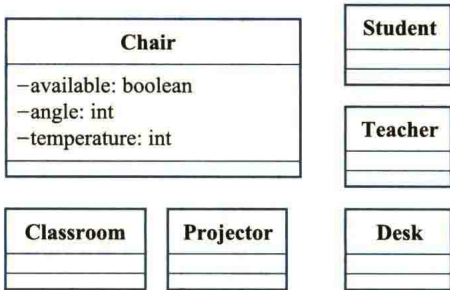


图 3.7 尚未开发完成的初始概念类图

3.2.3 概念间关系定义

仅知道一个系统中究竟有哪些类是不够的,类之间需要通过关系形成结构,有了结构才能在其基础上构建相应的功能,而不同的结构支持了不同的功能。

与之类比的是:通过本课程的前序课程“数据结构”,读者应了解不同的数据结构决定了其上可执行的不同操作。例如,顺序表可以执行折半查找,而链表则不行。

同样地,类和类之间的关系种类及性质,也决定着相关的类能否执行某些系统操作或行为职责,例如,两个类之间如果不存在关联关系,则一个类就无法创建、更新或调用另一个类。此外还决定相关类之间实例及其连接(link)是否合法,例如,两个类之间关系的重数约束为一对一(考虑在银行储蓄场景中,一个客户仅能在银行开立一个户头),则两个类的实例(对象)之间若存在一对五的情形(考虑在银行储蓄场景中,客户张三在招商银行开立了五个户头)就是非法的。

因此,需要基于初始概念类图继续定义类间关系。关系主要通过类型和性

质进行定义,常见的关系类型包括(一般)关系、泛化关系、聚合/组合关系。

(1) 关系:被描述为类之间的一般性的语义联系,通常抽象自相关类的实例之间有意义或值得关注的连接。在 UML 中,关系被表示为类之间的连线,如前文中图 3.5 所示的“教学”关系。关系可以通过一些性质来刻画,包括关系的度、重数、名称、角色等。

(2) 关系的度:表示参与关系的类的数量。一元关系即参与某个关系的类的数量只有一个。例如,离散数学中定义的反自反关系就是一种一元关系;又如,人和人之间的婚姻关系也是一元关系(因为只有一个类“人”参与——虽然婚姻关系形成时有两个实例参与)。二元关系即参与某个关系的类的数量有两个,这是系统中最为普遍的一种关系,如前文中图 3.5 所示的“教学”关系就是一种二元关系。其他还有三元关系等。

(3) 关系的重数(multiplicity):约束一个类与另一个类产生关系时的各自实例数量。重数一般有如图 3.8 所示的表示法。

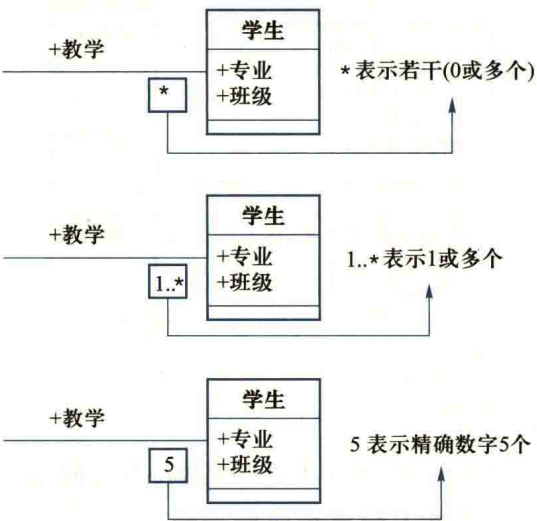


图 3.8 重数的表示法举例

重数是概念类图中非常关键的领域约束。正如前文所举的银行储蓄案例中所描述:如果“客户”类与“银行账户”类之间的重数是 1 vs. 1,则明确了其实例之间的连接需要满足这个约束关系。若存在某个客户(张三)开设了 5 个银行账户,则违背了约束,是非法的连接。同时,对“客户”类与“银行账户”类之间的重数分别标注为 1 vs. 1、1 vs. * 及 * vs. *,其业务功能实现时的复杂度以及开发成本等是不同的——显然其复杂度和成本逐渐升高。

因此,如果概念类图中明确了某个关系值得关注,并显式地标注了出来,那么明确标注这个关系的重数是非常必要的,否则就有需求模糊的嫌疑。

(4) 关系的名称:可以给关系命名,以辅助各利益相关方理解关系的意义。例如,图 3.8 中的“教学”就是关系的名称。需要注意的是,关系命名一般用动词短语,用以表达类之间有意义的关联。此外,关系命名时还可以用导向箭头表达

关系的方向,如图 3.9 表达“婚娶”关系中由新郎婚娶新娘(如果是“婚嫁”关系,导向箭头则需要反过来)。

(5) 关系的角色:类之间产生关系时,其各自担任的角色也可以显式地描述出来,以方便理解。尤其是在一元关系中,若不明确角色,则理解上是存在一定障碍的(如图 3.9 左侧)。当然,如果类名很显著地蕴含了角色,则不必再显式给出角色的描述。

实际上,基于角色标注的一元关系还可以借助“继承”关系来构造并转化为二元关系:以图 3.9 为例,可以根据角色名“新娘”和“新郎”分别定义一个类,让这两个新定义的类来继承“成年人”这个类;此时,“婚娶”关系便可直接标注在“新娘”和“新郎”这两个类之间,构成一个二元关系(如图 3.10 所示)。在后续设计环节,二元关系比一元关系的理解 and 处理更为便捷。

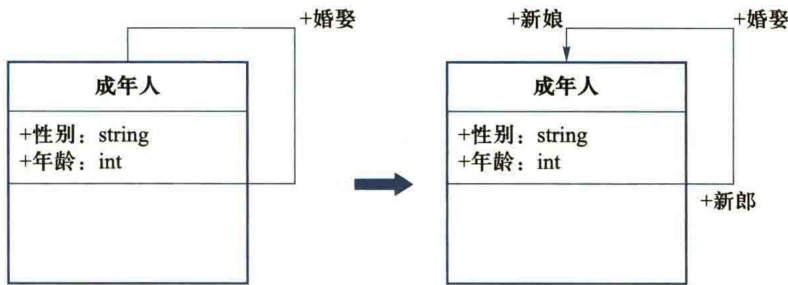


图 3.9 关系的角色描述举例

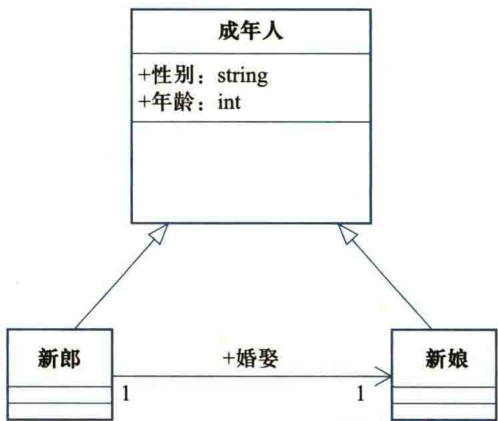


图 3.10 泛化关系举例

(6) 泛化 (generalization) 关系:UML 将泛化关系定义为特殊类(子类)与一般类(超类)之间的分类学关系。特殊类(子类)与一般类(超类)之间存在着“子集-超集”关系(即任意子类的实例均属于超类)。特殊类(子类)拥有一般类(超类)的属性。泛化关系与面向对象编程语言中的“继承”机制存在相通之处。泛化关系的 UML 表达如图 3.10 所示。

(7) 聚合 (aggregation)/组合 (composition) 关系:聚合表达了一种一般性的“整体-部分”关系;组合则是一种“强聚合”,它在表达“整体-部分”关系时蕴含

了:① 在某一时刻,一个“部分”的实例总是属于一个“整体”的实例(部分不会在多个整体之间分享);② “部分”总是属于“整体”(不存在游离在整体之外的部分);③ “部分”的生命周期囊括在“整体”的生命周期之内,即“部分”由“整体”负责创建和销毁,且一旦“整体”不存在,“部分”也就同样被销毁。在 UML 表示法中,聚合关系用空心菱形表示,组合关系用实心菱形表示,如图 3.11 所示。

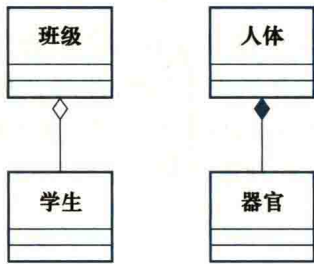


图 3.11 聚合/组合关系举例

现代软件工程实践表明:组合优于聚合。其原因在于聚合被认为是一种模糊的“整体-部分”关系,而作为“强聚合”的组合关系,对“整体-部分”关系的刻画明显要明确得多。

3.2.4 概念类图的开发过程

概念类图的开发过程如下。

第一步:发现概念与属性(详见 3.2.2 节)

第二步:甄选概念与属性(详见 3.2.2 节)

第三步:开发初始概念类图(详见 3.2.2 节)

至此,获得未标注关系的初始概念类图,接下来标注概念之间的关系。

第四步:识别关系的种类(详见 3.2.3 节)

除了一般性的关系之外,增补可能存在、重要且有意义(有意义即与当前系统的业务主题相关)的泛化关系,以及识别并标注重要且有意义的组合关系。

之所以强调“重要且有意义”,原因在于软件工程的一些实践并不建议标注所有与当前系统的业务主题相关的关系,以免形成过于复杂的蛛网。

对于这种观点,本书作者持谨慎态度,尤其是在模型驱动方法中:作者期望能基于用例粒度的合理划分来控制问题规模;并基于各个用例,构建各自的 概念类图——当然,不同用例之间若存在相同概念,则需要进行全局统一;只要用例大小划分得当,其对应的概念类图的关系复杂度应该在可控的范围内。这一做法的好处在于能够得到更精确的需求规约,有助于提升后续设计的可验证性,进而获得更可信的系统。

第五步:明确关系的度(详见 3.2.3 节)

根据业务主题相关的原则,明确类之间存在的二元关系(或三元关系),以及某个类上蕴含的一元关系。

第六步:给出关系的名称和角色(详见 3.2.3 节)

第七步:给定关系的重数(详见 3.2.3 节)

以 JPetStore 网站为例,可将相关用例的概念类图开发如下。

- 注册账号(如图 3.12 所示)

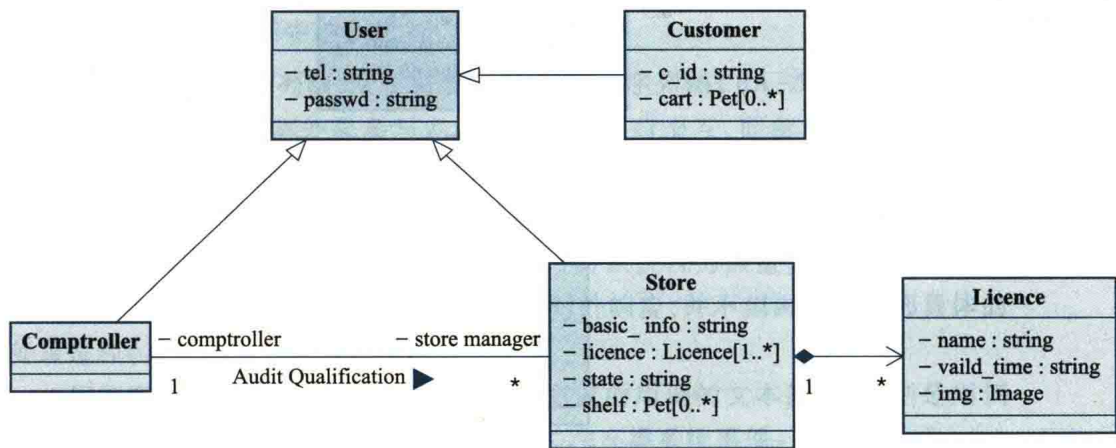


图 3.12 注册账号用例的概念类图

- 浏览商品与添加购物车(如图 3.13 所示)

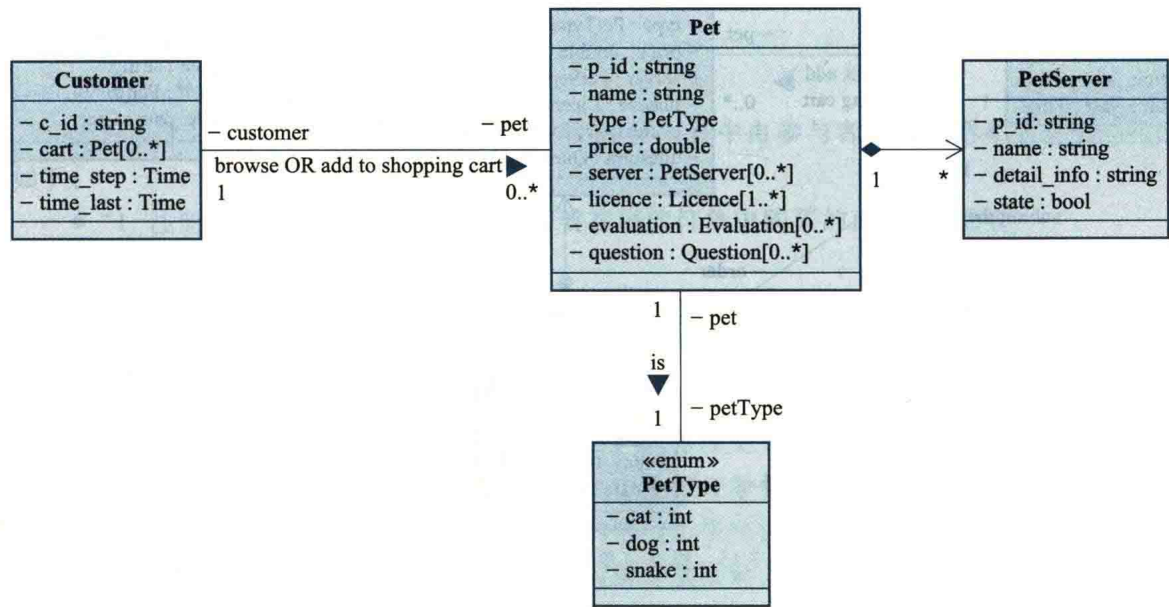


图 3.13 浏览商品与添加购物车用例的概念类图

- 上架商品(如图 3.14 所示)
- 提交订单(如图 3.15 所示)

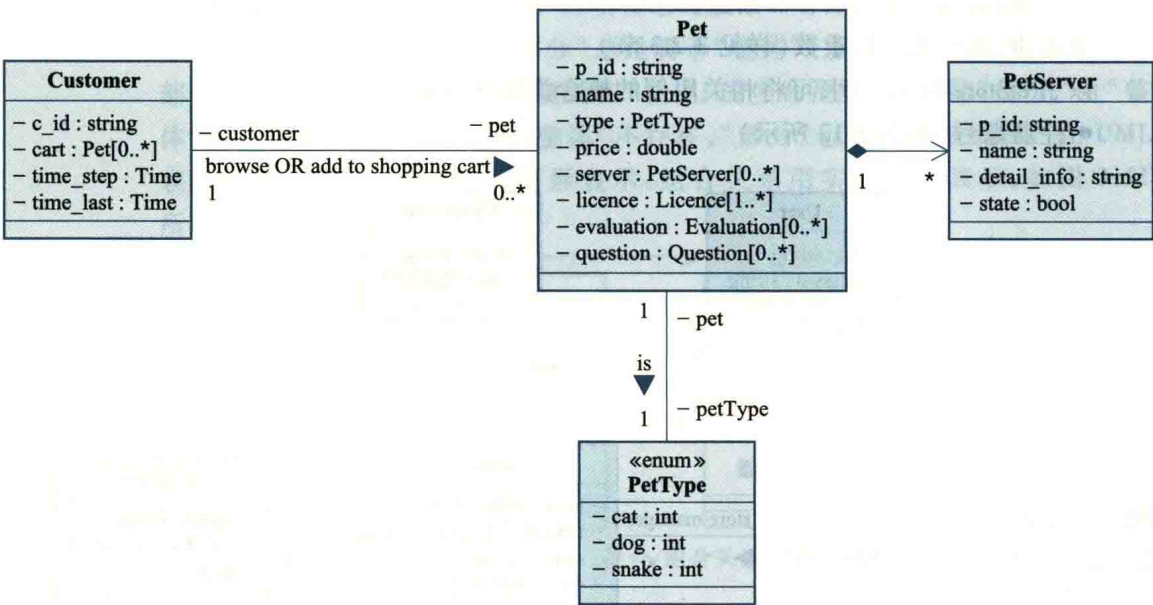


图 3.14 上架商品用例的概念类图

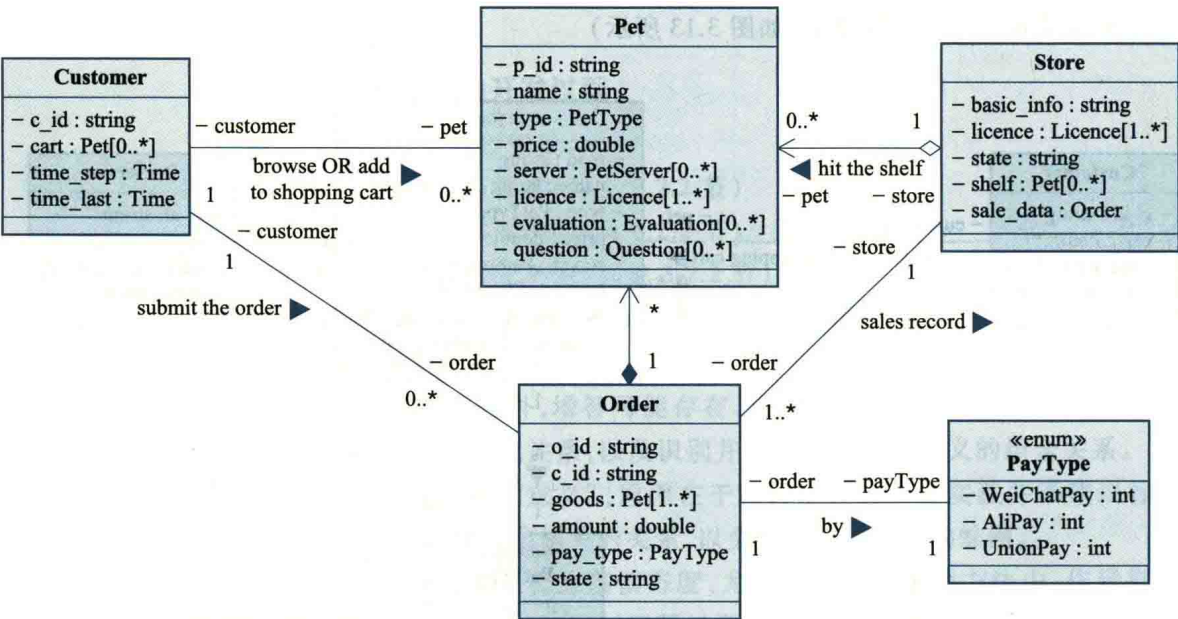


图 3.15 提交订单用例的概念类图

3.3 系统行为建模(RA-5)

用例的行为建模包含对用例所蕴含的参与者与系统之间交互过程的刻画，其中包含对系统操作的识别，以及基于系统操作所构建的用例内部参与者与系统之间的交互图。系统操作代表了系统应提供的服务(类似于方法代表了类所

能提供的服务),每个系统操作都将在后续的设计环节利用对象交互图来具体展现系统内部的对象是如何交互以完成该系统操作的。在面向对象方法中,交互图可以使用通信图或序列图来表达;本书在分析与设计环节,均选择使用序列图来表达(事实上,得到了序列图就可以非常容易地转换为通信图)。

3.3.1 识别用例中的系统操作

UML 只定义了序列图,而没有特别定义用例序列图(RA-5)。但系统的利益相关者总是期望了解外部参与者如何与系统进行交互,即参与者发起了某种系统事件,而系统是如何通过系统操作进行处理和响应的。用例序列图则是应用于这样的情形,它针对某个用例场景,通过表达参与者与系统之间的交互,以刻画系统的行为(系统能够做什么)。此时可将系统视为黑盒,即针对某个系统事件,只在系统序列图中刻画系统整体性的对外响应,并不刻画系统内部具体的处理过程(系统具体怎么做)。

用例序列图的输入是在上一节中开发完成的用例的文本描述,特别是用例的“主要过程”描述部分。系统序列图主要关注三类系统事件:

- (1) 参与者发起的外部事件(如“主要过程”描述中的参与者的请求);
- (2) 与时间相关的事件(如定时执行或周期执行的任务);
- (3) 错误或异常事件(如“扩展事件”中描述的某些事件)。

【案例 3.5】 以“管理订单”用例的详细文本用例作为输入,其系统操作的识别步骤如下。

第一步:分析由参与者发起的外部事件

分析表 3.11 中的“管理订单”用例的“主要过程”,其中由参与者发起的活动如下:

- “1. 注册用户从购物车中再次选择需要在当前订单中购买的商品,启动商品购买结算流程”
- “3. 当前用户确认物流服务商的选择,并从自己预留的物流地址列表选择一个地址,作为当前订单中的商品期望送达的地址”
- “5. 当前用户选择确认支付方式,并确认发票信息,发票信息包含发票类型、发票抬头、收票人信息、发票内容等。随后用户选择提交支付”
- “7. 各商品的销售商在获得系统通知后,将按照订单物流要求,通过订单选定的物流服务商发运商品,并获得各自商品的物流运单号提交给系统”

从上述描述中,不难发现一些需要系统响应的外部事件,具体如下:

- 确认商品订单;
- 确认物流选择;
- 确认发票信息;
- 追踪订单物流。

当然,在上述外部事件的提炼过程中,可以用“参与者活动”为线索,将其与对应的“系统响应”关联起来一起抽取和总结,从而使上述外部事件的提炼和命

名更加准确。

例如,仅从“7.各商品的销售商在获得系统通知后,将按照订单物流要求,通过订单选定的物流服务商发运商品,并获得各自商品的物流运单号提交给系统”的描述中,读者很难抓住外部事件的重点:“发运商品”与“提交运单号”,到底选择哪个事件作为系统操作呢——事实上两个都不合适。因为“发运商品”这个操作实际是由第三方的物流供应商这个外部系统来完成,而“提交运单号”这个事件作为操作的粒度太小而无法涵盖这一环节的业务内容。

当结合其对应的“系统描述”——“系统在收到当前订单对应各商品的运单号后,将各商品同运单号在系统中关联起来。同时系统会根据运单号请求物流服务商返回并载入本订单所对应各商品的相关物流运输细节……”,很容易提炼出“追踪订单物流”这个事件。

而抽取“确认商品订单”这个事件,同样也是基于联合“1.注册用户从购物车中再次选择需要在当前订单中购买的商品,启动商品购买结算流程”和“系统首先生成当前订单的表单,进入填写订单环节”两个信息进行综合提取的。

需要特别指出的是:“确认发票信息”事件触发了系统响应,使其在内部完成“更新运费—更新合计—修改订单支付状态”等一系列内部活动。不过,这些系统响应外部事件请求时的内部活动是无须当作系统操作进行处理的。

第二步:分析与时间相关(定时、周期)的事件

事实上,表 3.11 中的“管理订单”详细文本用例的“主要过程”描述中并未涉及“定时”“周期”等相关事件。

但读者可以设想如下场景:假如当前这笔订单是同时为多个身处不同地域的客户订购的商品,此时就需要明确在“确认物流”这个系统操作上发生了周期性的事件,即“确认物流”操作需要被循环多次。

第三步:分析扩展事件描述中的错误或异常事件

从表 3.12 的描述中可知,在当前订单所购商品期望送达的地址不在系统可选的物流地址列表中时,需要新增物流地址。因此,“新增物流信息”应该被识别为系统操作,且该操作应该包含在“是否存在期望物流地址”这个选择条件之中。

基于上述分析步骤,可以得到表 3.13 中系统操作的识别结果:

表 3.13 “管理订单”用例的系统操作

系统操作	备注
确认商品订单	
确认物流选择	循环包含(场景:为多个身处不同地域的客户订购的商品)
确认发票信息	
追踪订单物流	
新增物流信息	条件包含(检验“是否存在期望物流地址”时,条件取“否”)

至此,系统操作识别完毕,接下来探讨用例序列图的具体开发过程。

3.3.2 开发用例序列图

如上文所述,UML 并未特别定义应用于参与者与系统之间交互的序列图,可直接沿用通用的序列图定义进行表达。

本文所使用的通用 UML 序列图的图形元素来自 OMG UML 规格说明书 (specification version 2.5.1)中关于序列图(sequence diagram)的定义。通用 UML 序列图一般包括如下常用图形元素:参与者、对象、生命线、激活期、消息和控制段。

作为用例序列图,可以暂时不必探索系统这个黑盒子内部的对象构成以及对象之间的消息传递,只需刻画角色与系统(或用例代表的功能特性)之间的交互行为。因此用例序列图的常用元素为:参与者、系统(或用例)、生命线、激活期、系统操作(消息)和控制段。下面以图 3.16 为例介绍用例序列图的上述各个元素。

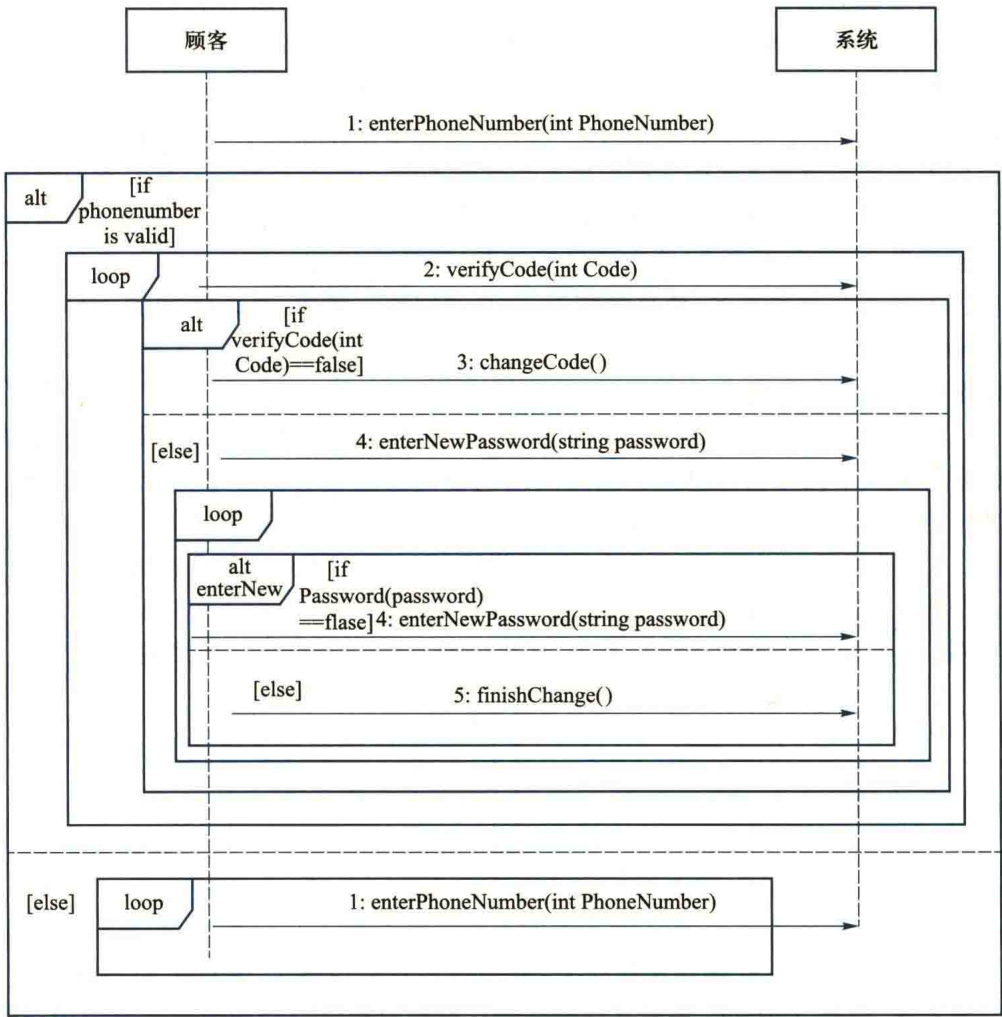


图 3.16 用例(修改用户密码)序列图示例

● 参与者:在用例图中,参与者用火柴人表示;在早期的序列图中也是如此。但最新的 OMG UML 规范不再使用这个图形元素,而是统一使用矩形框予以表示(见“顾客”矩形框)。

● 系统(或用例):使用与对象序列图(请参阅“系统设计”相关章节)中“对象”相同的图形元素(即矩形框)予以表示(见“系统”矩形框)。需要注意的是,一般会将待开发系统的名字(或用例的名字)填入该矩形框,代表其为参与者相关服务请求的响应者;同时需要留意,此时并不拆开系统(或用例),探寻其内部的具体处理和响应过程(这部分工作在设计环节构造“对象序列图”时予以完成),只是将其当作一个响应参与者相关业务服务请求的黑盒子。

● 生命线(lifeline):生命线代表用例序列图中的参与者或系统的存在期。它位于用例序列图中参与者或系统矩形框底部中心,用一条垂直的虚线表示,而参与者与系统交互过程中所产生的消息,均存在于两条虚线之间。

● 激活期(activation):激活期代表用例序列图中,系统为响应参与者发出的一项服务请求(消息)而获得控制权的时期。它位于用例序列图的生命线上,用长条矩形来表示(矩形的长度也代表了激活期的时间跨度)。

● 系统操作/消息(system operation/message):消息在对象序列图中,代表的是对象之间的方法调用;而在用例序列图中,它的对应概念是系统操作。系统操作代表的是参与者发出的一项需要系统处理和响应的服务请求。它位于用例序列图中参与者与系统的生命线之间,垂直于生命线,用直线箭头来表示;箭头由消息发送者(服务请求者)指向消息接收者(服务响应者);箭头上标注消息的签名(signature),即消息名、消息参数表、消息返回值。例如,图 3.16 中的 enterPhoneNumber(int PhoneNumber)即包含了消息签名中的消息名与消息参数表,其下的箭头即该签名所代表的消息。

● 控制段(fragment):在结构化程序中,通常使用顺序、选择、循环等控制结构来刻画程序流程;类似地,为了刻画用例内部所蕴含的动作序列,也可以引入相应的控制结构,称之为控制段(或直接简称“段”)。常用的控制段有(见图 3.17)选择段(“alt…else…”段)和循环段(“loop…”段)。其表达方式为用矩形框框住需要选择或循环的系统操作(消息),在左上角用缺角矩形标注控制段类别(选择 alt 或者循环 loop);其中,还可以通过[…]来标注选择或循环条件。

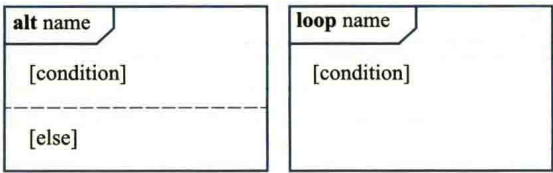


图 3.17 选择段(左)和循环段(右)

需要注意的是,在结构化程序中,常见的选择结构除了“if-else”双分支选择结构之外,还有“case-do”多分支选择结构。但在对象序列图和用例序列图中,均不用考虑多分支选择结构(旨在减少序列图的模型元素种类)。当然,还有此处

的“loop”循环代表的是“while-do”循环,而结构化程序中常见的“do-until”循环结构在对象序列图和用例序列图中也不用考虑(基于多分支结构类似的原因)。尽管如此,当前的选择段和循环段仍然可以表达出多分支选择结构和“do-until”循环结构(例如,多分支选择结构可以利用多个双分支结构方便地表达出来),只是在序列图中采取了最简的模型元素的集合^①。

具体的用例序列图绘制过程如下。

(1) 阅读并理解详细用例描述,尤其是理解 system response 部分及其某个 response 所对应的 actor action。

(2) 从 action-response 对中抽象出 system operations(一般是动宾短语),表示某类参与者期望系统完成某项业务功能时需要执行的业务操作。同时注意取名要有特定意义(自明性),例如,使用 enterNewPassword 就要比 enterASerialOf-Charaters 更合适。

(3) 为识别到的系统操作增加参数,表示某类参与者通过系统来完成某项业务操作时,系统需要从参与者那里获取的数据。例如,在执行新密码设置操作时,enterNewPassword()需要参与者输入一个新的字符串作为新设置的密码,因而有 enterNewPassword(string password)。

(4) 仔细阅读详细文本用例的主要流程,根据业务流程的实际处理步骤,绘制用例序列图(明确用例序列图的图像元素——尤其注意 loop 和 alternative);绘制时注意根据业务流程的实际执行逻辑,增补必要的细节(这些细节——尤其是重要细节需要向用户进行确认)。

3.4 系统操作契约开发(RA-6)

本节讲述需求分析阶段最后一个重要的产出物“系统操作契约”的开发过程。系统操作契约定义了系统操作执行前后导致系统状态的变化(对象被创建或销毁、对象属性被更新、对象之间的连接关系被创建或销毁)。系统操作是导致系统状态发生变化的原因,因此为了刻画系统操作契约,需要首先了解系统状态变迁的本质,继而通过系统操作执行前后的状态描述,来刻画系统操作契约的主要内容。

3.4.1 系统状态变迁的本质

在构建用例序列图之前,读者需要对软件系统状态及状态变迁的本质形成一种新的认识。

(1) 从本质上讲,在以面向对象方法开发的系统中,系统是由一组相互连接的对象所构成,它通过对象间的交互来产生自身的行为并完成系统用户请求的业务目标。

^① 这也是符合“奥卡姆剃刀”原则的。

(2) 其中,系统中所包含的对象的集合、各对象本身的状态(即对象属性的取值),以及对象之间的连接关系,构成了该系统当前的状态。

(3) 系统状态的变更则由对象之间的消息传递(即方法调用)来触发,对象间发生消息传递的基本前提是对象之间存在连接关系,连接关系的种类包括一般关系、泛化-继承关系、聚合/组合关系等;对象间发生消息传递的结果,会导致系统的状态发生如下三类变化:

- ① 新对象被创建或原有对象被销毁;
- ② 对象自身的属性值发生改变;
- ③ 对象间新的连接关系被创建或对象间原有的连接关系被销毁。

基于上述新的系统认识,读者在细化用例的文本描述时,就可针对性地从对象、对象属性、对象间关系的视角来刻画用例内部的动作序列。

3.4.2 前/后置条件开发

系统操作契约主要定义系统操作执行前后导致系统状态的变化(对象被创建或销毁、对象属性被更新、对象之间的连接关系被创建或销毁)。

开发系统操作契约的核心工作是构造霍尔三元组^[24]中的 Pre-condition(前置条件)和 Post-condition(后置条件)。

霍尔三元组可表达为:

$$\{ \text{Pre-condition} \} m() \{ \text{Post-condition} \}$$

其中,m()代表系统操作。围绕该系统操作的前/后置条件的含义如下。

- Pre-condition(前置条件):定义系统状态在系统操作执行之前必须满足的假设条件。例如:哪些对象在执行之前必须存在,哪些对象的某些属性的取值应该满足什么要求,哪些对象之间存在连接。
- Post-condition(后置条件):定义系统操作完成后,当前系统发生状态变化后所应呈现的结果。这些状态变化后的结果包括:哪些对象被创建或销毁,哪些对象属性被更新,哪些对象之间的连接关系被创建或销毁。

基于霍尔三元组,可以定义系统操作的契约。为便于系统操作契约的开发,可使用模板(见表 3.14)进行辅助,其包含了前/后置条件加上一些基础信息。

表 3.14 系统操作契约模板

系统操作	当前契约所属的系统操作
交叉引用	当前系统操作所属的用例
前置条件	系统操作在执行之前必须满足的假设条件(如上文)
后置条件	系统操作引发系统发生状态变化后所应呈现的结果(如上文)

系统操作契约定义的实例如表 3.15 所示。

表 3.15 用户/商户注册系统操作契约

系统操作契约	
系统操作	sign_up(tel:string,code:string,password:string,type:string)
职责	验证手机验证码并注册设置账号和密码
交叉引用	注册账号(用例)
备注	3 s 内完成验证和注册响应
异常处理	无
前置条件	1. 验证码格式合法,否则提示 2. 电话号码格式合法,否则提示 3. 密码格式合法,否则提示
后置条件	1. 如果通过,且选择顾客,则创建顾客对象,并写入数据库 2. 如果通过,且选择商家,则创建商家对象,并写入数据库

注:本表包含了比表 3.14 模板更多的契约组成部分,这些部分若无特殊需要可以忽略。

3.5 需求管理

前面四个小节论述了模型驱动方法中需求分析阶段中最核心的建模过程。在需求工程中,除了需求分析之外,还需要开发需求文档,并实施需求管理及需求评审等活动。

3.5.1 需求开发过程

- (1) 确定项目背景和目标:了解项目所处的背景和目标,包括项目的整体目的、范围和约束条件。
- (2) 收集需求信息:通过与利益相关者交流、实地调研、文档分析等方法,收集项目的需求信息。可以采用面谈、问卷调查、观察等方式与相关人员进行沟通,确保全面了解需求。
- (3) 分析需求:对收集到的需求进行仔细分析,将其整理成可操作的需求清单。识别和记录功能需求、非功能需求、优先级和相关的约束条件。模型驱动方法所需要开发的需求分析模型在此阶段可以实施。
- (4) 确定需求的准确性和一致性:与利益相关者验证和确认他们的需求,确保需求的准确性和一致性。这可以通过召开会议、组织讨论或其他形式与相关人员进行验证。
- (5) 编写需求文档:根据分析和确认的需求,编写需求分析说明书。此文档应包括项目背景、目标、利益相关者、功能需求、非功能需求、优先级、需求变更控制等内容。
- (6) 审查和确认:将需求分析说明书交给利益相关者进行审查和确认,根据反馈进行必要的修改和调整。

(7) 更新和维护:随着项目的进展,需求可能会发生变化。因此,需求分析说明书应随时进行更新和维护,确保与项目一致。

3.5.2 需求文档开发

模型驱动开发的 RA-1~RA-6 将成为需求分析阶段的最重要 (first-class) 的软件制品,同时会作为下一阶段系统设计部分的输入 (尤其是 RA-4~RA-6)。但作为需求分析阶段构成系统用户与系统开发组之间共识的一份技术协议书,软件需求规格说明书 (software requirement specification, SRS) 是这一阶段的里程碑式产出物,是后续设计工作的基础和依据,也是测试和验收的依据。

一般而言,在同一个团队中,SRS 需用统一格式的文档进行描述。SRS 可遵循国标 GB/T 9385—2008《计算机软件需求规格说明规范》,或者遵循国际标准 ISO/IEC/IEEE 29148:2018 *Systems and software engineering—Life cycle processes—Requirements engineering*,以及工业界较常使用的 RUP、Volere、SERU 等软件需求规格说明书模板 (SRS template) 进行开发。需要说明的是,在工业界的实际应用中,使用者往往根据自己的需要,对这些模板进行裁剪。国标 GB/T 9385—2008 建议,一份良好的 SRS 可参考如下模板进行编制:

-
- 1. 引言
 - 1.1 目的
 - 1.2 范围
 - 1.3 定义、简写和缩略语
 - 1.4 引用文件
 - 1.5 综述
 - 2. 总体描述
 - 2.1 产品描述
 - 2.2 产品功能
 - 2.3 用户特点
 - 2.4 约束
 - 2.5 假设和依赖关系
 - 2.6 需求分配
 - 3. 具体需求
 - 3.1 外部接口
 - 3.2 功能需求
 - 3.3 性能需求
 - 3.4 数据库逻辑需求
 - 3.5 设计约束
 - 3.6 软件系统属性
 - 3.7 具体需求组织
 - 3.8 附加说明
 - 4. 附录
 - 5. 索引
-

本章小结

本章从用例开发开始,逐一展现了需求分析过程的各项任务,包括用例的识别与开发、系统行为建模、系统结构建模、系统操作契约开发等。同时,针对需求阶段生成的几种重要制品进行了描述,包括用例图、高层文本用例、详细文本用例、用例序列图、概念类模型、系统操作契约等。此外,本章描述了各个制品之间存在的逻辑联系。

本章习题

思考与实践

1. 请尝试为 ATM 系统(或其他任意自选系统)开发一个用例图。要求:
 - (1) 至少识别 4 个用例;
 - (2) 至少包含 2 个参与者;
 - (3) 至少使用 3 种用例图的关系。
2. 请尝试为 ATM 系统(或其他任意自选系统)中的任意一个用例开发其文本用例。要求:
 - (1) 首先开发其高层文本用例;
 - (2) 基于高层文本用例开发其详细文本用例;
 - (3) 需包含扩展流程的定义。
3. 请尝试为 ATM 系统(或其他任意自选系统)中的任意一个用例开发其概念类图。要求:
 - (1) 至少定义 5 个类,每个类至少有 1 个关键属性;
 - (2) 至少涉及 3 种类间关系,且关系性质(包含关系的度、重数、命名、角色等)定义清晰。
4. 设想为上课的教室开发一个设备管理系统:
 - (1) 请和周围的学习伙伴通过“头脑风暴”,明确它应该建设的样子,并将讨论结果记录下来作为粗略的“用户需求”;
 - (2) 请使用用例建模技术来识别并开发用例图,并分别使用高层和详细文本用例来刻画每个用例的活动序列;
 - (3) 请使用概念建模技术来构建每个用例的概念类图;
 - (4) 请识别用例中的系统操作,并开发用例序列图;
 - (5) 请针对用例序列图中的每个系统操作,定义其操作契约。
5. 设想为附近的超市开发一个线上线下联合运营的超市系统:
 - (1) 请和周围的学习伙伴通过“头脑风暴”,明确它应该建设的样子,并将讨论结果记录下来作为粗略的“用户需求”;
 - (2) 请使用用例建模技术来识别并开发用例图,并分别使用高层和详细文

本用例来刻画每个用例的活动序列；

- (3) 请使用概念建模技术来构建每个用例的概念类图；
- (4) 请识别用例中的系统操作,并开发用例序列图；
- (5) 请针对用例序列图中的每个系统操作,定义其操作契约。

6. 自选主题并完成需求分析与建模：

(1) 请与周围的学习伙伴通过“头脑风暴”,明确它应该建设的样子,并将讨论结果记录下来作为粗略的“用户需求”；

(2) 请使用用例建模技术来识别并开发用例图,并分别使用高层和详细文本用例来刻画每个用例的活动序列；

- (3) 请使用概念建模技术来构建每个用例的概念类图；
- (4) 请识别用例中的系统操作,并开发用例序列图；
- (5) 请针对用例序列图中的每个系统操作,定义其操作契约。

综合实验一

一、实验目标

1. 识别用例
2. 开发用例图
3. 编写用例描述
4. 识别用例系统操作
5. 开发用例序列图
6. 识别初始概念类
7. 开发概念模型
8. 开发操作契约

二、实验学时

6 学时。

三、实验步骤

1. 识别用例

以项目小组的形式选择目标项目,提炼出目标系统的主要功能需求并予以描述。可以不断询问这样的问题——谁来使用这个系统?其目的是完成什么业务?了解相关用户与目标系统之间的系统级交互,举例如下。

通过交互式、非干扰式等需求获取方法,得到用自然语言描述的 JPetStore 在线宠物商店系统的用户需求描述如下:游客浏览不同类别宠物信息,选择查看感兴趣的宠物信息,包括宠物名称、描述、单价等;游客注册商店用户,填写用户、账户和个人偏好等信息;游客登录账户浏览宠物信息;注册用户添加宠物到购物车,填写购买数量;注册用户对购物车内商品进行更新,提交购物商品清单;注册用户填写支付信息(信用卡、微信、支付宝等)、收货信息,确认商品订单并提交后台处理;系统处理注册用户的商品订单……(其他内容省略)

运用课堂教学中所提到的参与者识别方法,包括参与者目标法、事件分析法等,从已经明确系统边界的需求描述文本中获取目标系统的参与者。这时绘制一张“参与者-业务目标”表将有助于完成这项工作。

有了这些参与者之后,即可从参与者使用系统的业务目标入手来识别用例。参与者使用系统所要达成的一个业务目标就作为一个候选用例而存在。运用课堂教学中提到的“老板测试”方法检测用例,并对违例情形进行适当判定,应能对所获得的候选用例集完成用例识别,并确定用例。通过分析 JPetStore 用户需求描述文本,获得候选用例集,进行用例识别,并确定用例如表 3.16 和表 3.17 所示:

表 3.16 “参与者-业务目标”表(用例识别后)

具体用户	代表角色(参与者)	业务目标
未注册用户	游客	浏览商品(√)
		注册(√)
注册用户	注册用户	浏览商品(√)
		输入用户名和密码(×)
		登录账户(√-违例条款)
		管理购物车(√)
		移除购物车商品项目(×)
		管理订单(√)
		处理支付(√)

表 3.17 “外部事件”表(用例识别后)

外部事件	源点/终点参与者	业务目标
外部系统被请求完成待开发系统的订单支付	【可选】信用卡支付系统	(协助)处理支付(√)
	【可选】微信支付系统	本次分析不涉及(×)
	【可选】支付宝支付系统	

2. 开发用例图

识别用例之后,可以使用工具软件创建用例图。通过刻画参与者与参与者之间的关系、用例与用例之间的关系,以及参与者与用例之间的关系来展现系统的功能。以用例图表示的 JPetStore 系统初始用例图如图 3.18 所示。

通过不断重复识别用例方法,发现其他用例后可补充完整的用例图,得知完整的系统功能特性。

3. 编写用例描述

通过开发用例图只能知道系统的主要功能,却无法得知用例内部的动作序列,而动作序列的发生则蕴含了具体的功能需求。为此,在完成用例图后需要编



图 3.18 用例图

写用例描述,共同构成第一个分析模型——用例模型。运用课堂教学中所提到的高层文本描述、详细文本描述方法,通过结构化的描述模板来完成用例描述的梗概和细化。表 3.18 和表 3.19 给出了 JPetStore 宠物商店中“管理订单”的用例描述。限于篇幅,详细文本描述的前置条件、后置条件、主要过程、扩展事件和非功能需求已省略。

表 3.18 用例“管理订单”的高层文本描述

主要元素	说明
1. 用例名称	管理订单
2. 业务目标	注册用户为购物车中选定的商品形成一份包含支付信息和物流信息的交易表单,并在支付后的订单中展现支付与运输细节及商品列表
3. 用例级别	业务功能级别
4. 主要参与者	注册用户
5. 过程摘要	用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付 用例执行过程:当用户决定支付时,系统会构造一个支付表单,表单包含待购商品列表及价格,并要求注册用户选择支付途径、提供支付信息并填写物流地址;付款完成时系统会创建一个订单,其中包括支付与运输细节及商品列表 用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看

表 3.19 用例“管理订单”的详细文本描述

主要元素	说明
1. 用例名称	管理订单

续表

主要元素	说明
2. 业务目标	注册用户为购物车中选定的商品形成一份包含支付信息和物流信息的交易表单,并在支付后的订单中展现支付与运输细节及商品列表
3. 用例级别	业务功能级别
4. 主要参与者	注册用户
5. 过程摘要	用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付 用例执行过程:当用户决定支付时,系统会构造一个支付表单,表单包含待购商品列表及价格,并要求注册用户选择支付途径、提供支付信息并填写物流地址;付款完成时系统会创建一个订单,其中包括支付与运输细节及商品列表 用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看
6. 前置条件	省略
7. 后置条件	省略
8. 主要过程	省略
9. 扩展事件	省略
10. 非功能性需求	省略
11. 其他	省略

4. 识别用例系统操作

通过构造用例模型,可以对来自用户、系统的需求进行完整、规范的描述,形成需求规格说明,为下一步开展系统分析和设计提供重要依据。在面向对象开发方法中,系统被认为是由一组相互连接的对象所构成,并通过对象间的交互来产生自身的行为,实现系统用户请求的业务目标。因此可根据系统的用例描述,有针对性地从对象、对象属性、对象间关系的视角来刻画用例内部的动作序列,用于说明外部参与者如何与系统进行交互,即参与者发起了某种系统事件,而系统是如何通过系统操作进行处理和响应的。

接下来将利用面向对象分析和设计的方法,完成 JPetStore 宠物商店系统的另外一个分析模型——行为模型。使用已开发完成的用例描述,尤其是用例的“主要过程”描述部分(见表 3.20),通过用例序列图表示外部参与者与系统之间进行交互的过程,即参与者发起了某种系统事件,而系统通过系统操作进行处理和响应。然后抽取和总结由参与者发起的外部事件,分析事件的时间、异常等因素。表 3.21 是 JPetStore 宠物商店“管理订单”用例的系统操作,其中从表 3.20 的外部事件 1、3、5、7 抽取系统事件,从外部事件 3 中还抽取因条件因素产生的系统操作。

表 3.20 “管理订单”用例的“主要过程”

参与者活动	系统响应 (What/How)
0. 用例开始于注册用户完成了商品浏览及商品向购物车的添加,并准备支付	
1. 注册用户从购物车中再次选择需要在当前订单中购买的商品,启动商品购买结算流程	2. 系统首先生成当前订单的表单,进入填写订单环节。订单表单将展现拟结算的商品列表,列表中的每一条记录显示了当前订单中需要购买的商品的基本信息、单价与数量、折扣等信息,并自动计算出小计金额和合计金额
3. 当前用户选择确认支付方式,随后用户选择提交支付	4. 系统根据当前用户的支付选择准备支付流程
5. 当前用户确认物流服务商的选择,默认值为自己预设的物流地址。如果预设物流地址并非当前用户期望的地址,则可新增物流地址	6. 系统根据当前用户对物流服务商及物流地址的选择确认,自动更新订单中的运费数额,并再次计算得到更新后的合计金额,且将订单状态置为“待支付”
7. 各商品的销售商在获得系统通知后,将按照订单物流要求,通过订单选定的物流服务商发运商品,并获得各自商品的物流运单号提交给系统	8. 系统在收到当前订单对应各商品的运单号后,将各商品同运单号在系统中关联起来。当前用户此时可查看订单详情,包括商品列表、物流信息、支付信息
9. 用例结束于注册用户在支付完成后对订单中有关支付与运输细节及商品列表的查看	

表 3.21 “管理订单”用例的系统操作

系统操作	备注
确认商品订单	
确认支付方式	
确认物流选择	
更改物流信息	条件包含(检验“是否更换收货地址”时,默认取“否”)
提交商品订单	

5. 开发用例序列图

作为用例序列图不必探索用例内部的对象构成以及对象之间的消息传递,只需刻画角色与系统(或用例代表的功能特性)之间的交互行为。如前所述,从

详细用例描述中的“参与者活动”-“系统响应”表格中,抽象出相应系统操作(一般是动宾短语),代表某类参与者期望系统执行的业务操作,完成相应某项业务功能。在本步骤中,为识别到的系统操作增加参数,表示上述业务操作执行时系统需要获取的数据。最后,根据业务流程的实际执行逻辑,增补必要的细节(必要时需要同用户进行确认)。

图 3.19 是 JPetStore 宠物商店某系统操作对应的用例序列图。

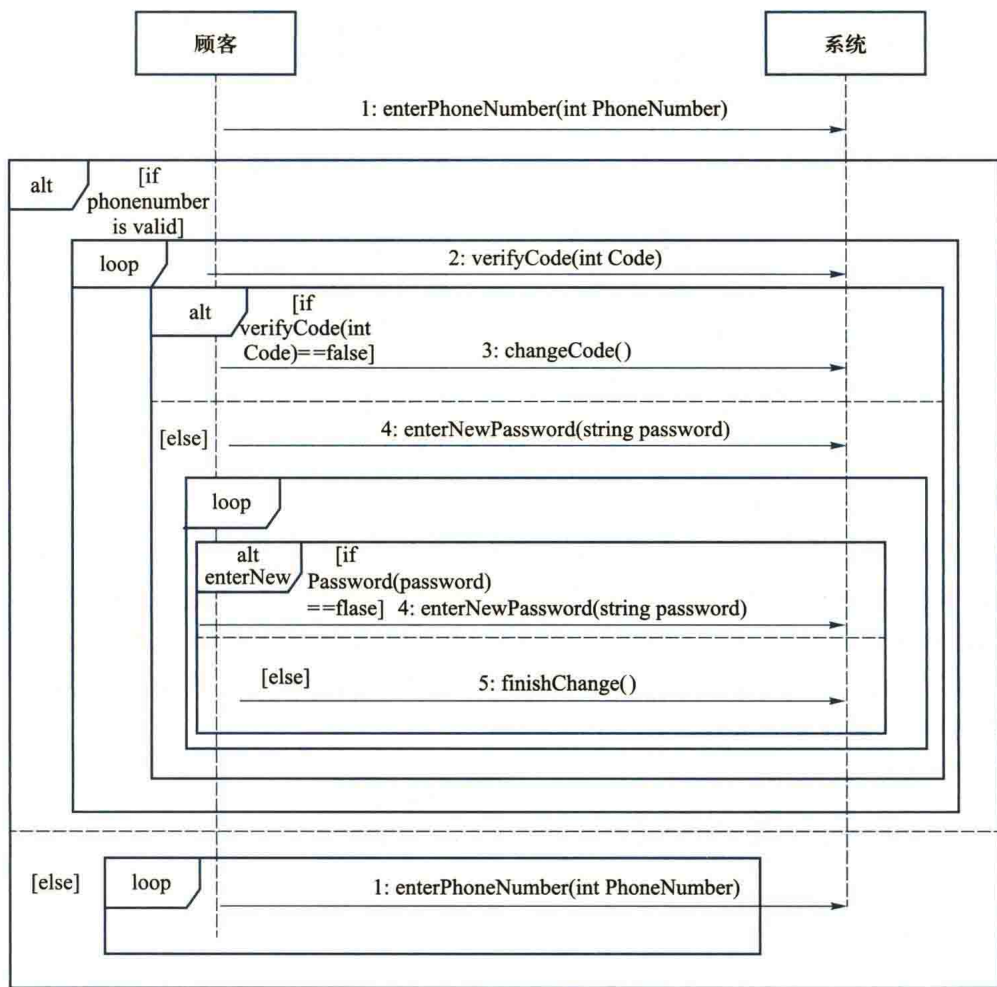


图 3.19 “注册账户”用例序列图(对应“修改用户密码”系统操作)

6. 识别初始概念类

在面向对象开发系统中,系统的所有功能都是通过相应的类来提供的。前期已经从用例文档中找出用例动作序列,构建了行为模型,达成了用例请求的业务目标。下一步需要建立最后一个分析模型——概念模型,明确业务目标的实现价值,识别出不同类别的概念类,以便行为模型中的系统行为分配到特定的对象中。

初始概念类使用类和类图对系统业务功能(操作)进行抽象和表达,并且暂时忽略其行为部分、概念之间的联系(关系),只包含概念(类名)、重要的数据信

息(属性)。首先从系统用例建模时生成的详细用例描述,以及系统行为建模时识别的系统操作中,围绕待开发系统构建的需要,选取其中相关的名词、对象加入候选概念(类)和属性集合。然后甄选出通过系统进行操作和控制的对象或者属性,将其纳入初始概念类及其属性的集合。图 3.20 是 JPetStore 宠物商店的初始概念类图。

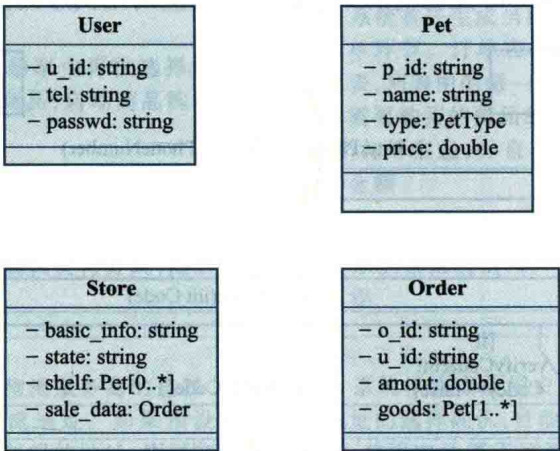


图 3.20 初始概念类图

7. 开发概念模型

构造初始概念类图的最终目标就是从系统的角度明确说明每一个初始概念类的职责和属性以及类之间的关系(包括关系的度、重数等),并根据这些视图来描述和理解目标系统,从而为后续设计阶段的系统开发提供基本的素材。图 3.21 至图 3.23 展示了 JPetStore 宠物商店实体用例的概念类图,图中涉及类、类的职责、属性和关系等内容。

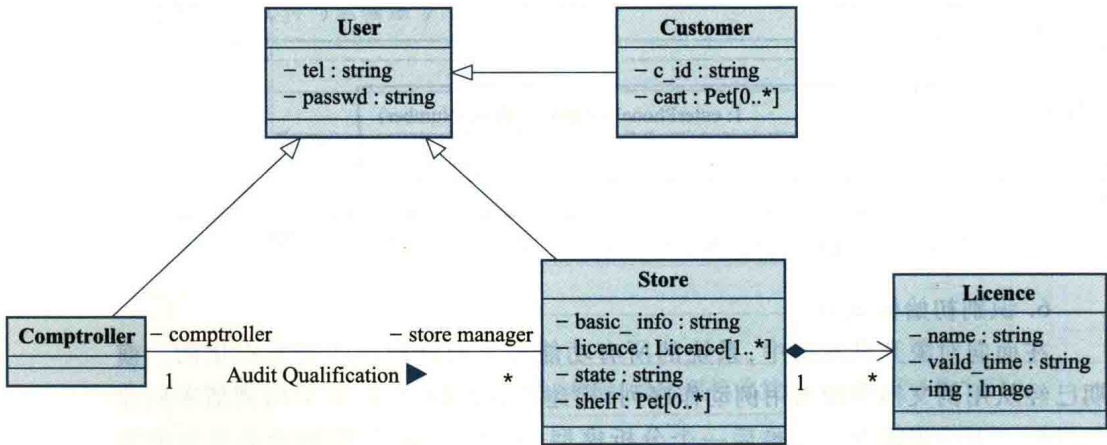


图 3.21 注册账号用例的概念类图

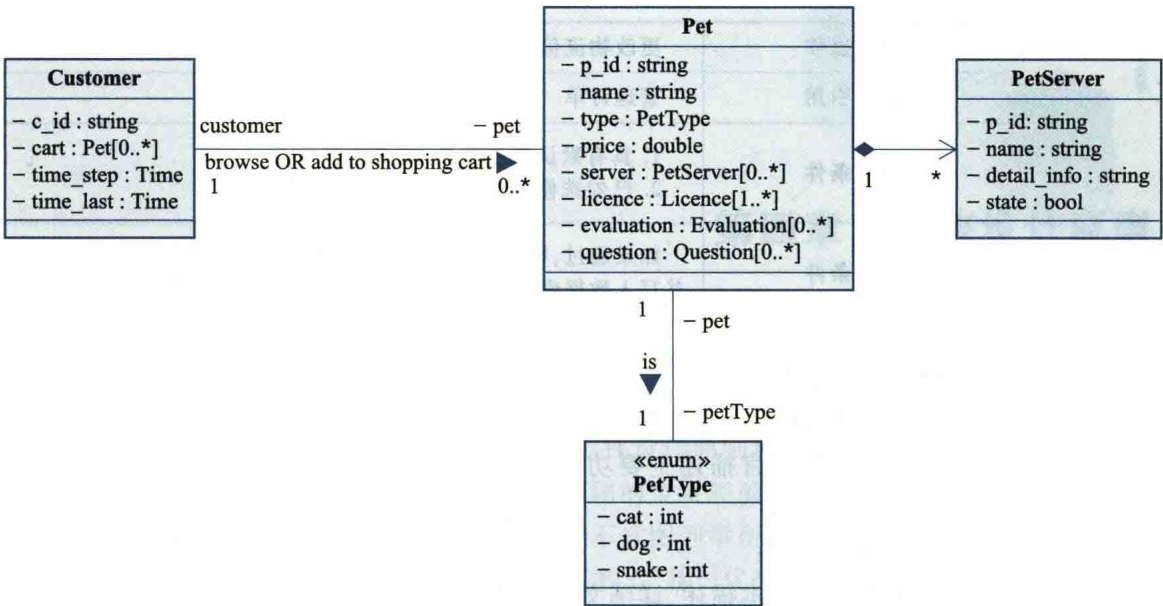


图 3.22 浏览商品与添加购物车用例的概念类图

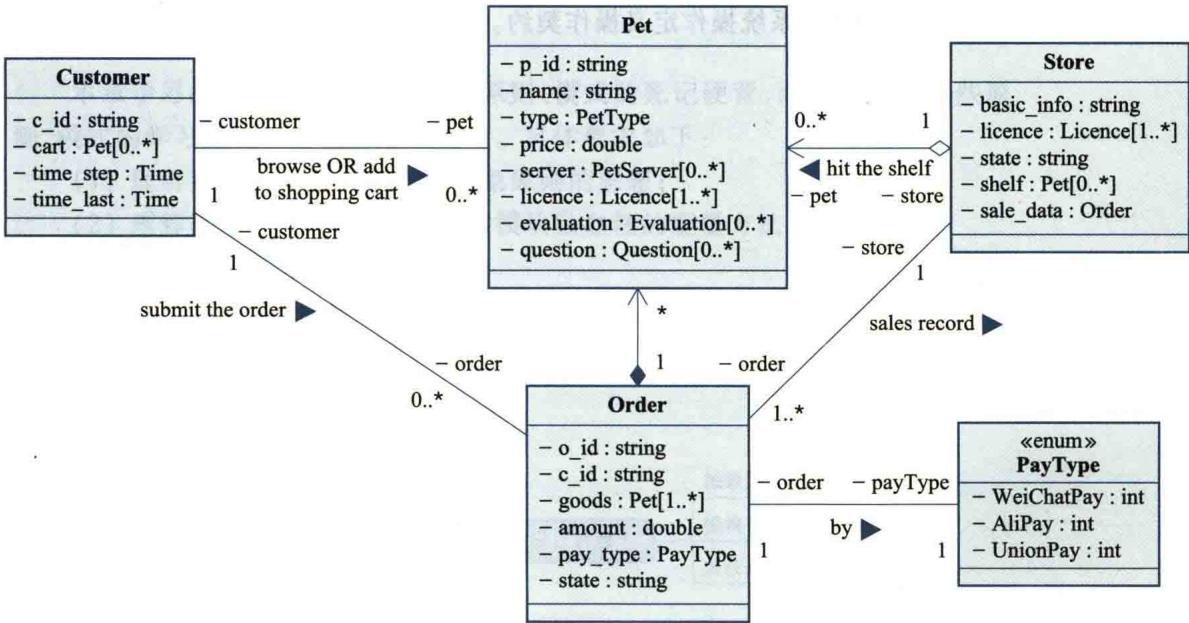


图 3.23 提交订单用例的概念类图

8. 开发操作契约

系统操作契约主要由系统操作执行前后的状态描述组成,其核心部分是其前置条件和后置条件。其中,前置条件代表了系统操作执行之前,需要满足的基本假设条件。后置条件代表了系统操作执行完毕之后需要满足的系统状态。表 3.22 是 JPetStore 宠物商店中“管理订单”用例的“更改物流信息”系统操作契约。

表 3.22 系统操作契约模板

系统操作	更改物流信息
交叉引用	管理订单
前置条件	1. 具有默认物流信息,否则提示 2. 已勾选修改物流信息选项,否则提示
后置条件	如果通过,则为当前订单创建物流信息,替换默认物流信息,并写入数据库

四、实验作业

对于所选目标系统,选用一种工具软件,每个项目小组完成以下工作:

- 1. 使用自然语言描述主要功能性需求,构造候选用例集,识别用例并给出理由;
- 2. 绘制用例图;
- 3. 编制高层文本描述、详细文本描述;
- 4. 识别系统操作并绘制用例序列图;
- 5. 为用例开发一个概念类图;
- 6. 为用例的系统操作定义操作契约。